Information Systems Frontiers (2025) 27:259–281
https://doi.org/10.1007/s10796-023-10436-z

Business-Process-Driven Service Composition in a Hybrid Cloud Environment

Jian Xu[1,2] · Hemant K. Jain[3] · Dongxiao Gu[1,4] · Changyong Liang[1,5]

Accepted: 23 September 2023 / Published online: 21 October 2023

Abstract
Cloud services have been widely used to support tasks in business processes. A variety of services with differing types, brands, and quality of service (QoS) characteristics are available from various vendors. Additionally, companies also build their own private clouds to meet specific business requirements related to performance, privacy, and security. The problem of selecting and assembling appropriate services to support an organization's multiple related business processes is very challenging. This problem also differs from traditional product/service selection problems because of the presence of business processes with non-sequential tasks and multiple, related business processes. The various QoS characteristics of services, the special requirements of some subtasks in the business processes, compatibility between cloud services, and the coordination of multiple business processes need to be considered when selecting appropriate services. This paper develops a multi-factor cloud service composition optimal selection (CSCOS) model to formalize the constrained combinatorial optimization problem and designs an improved differential evolution algorithm based on a constructive cooperative coevolutionary framework (C3IMDE) for solution. Experiments on synthetic data demonstrate that C3IMDE has better efficiency and stability than benchmark algorithms, especially for large-scale, multi-process collaborative optimization.

Keywords OR in service industries · Cloud service composition · Hybrid cloud environments · Multi-process collaborative optimization · Heuristics

1 Introduction

Developments in distributed computing, storage, and integration technologies have resulted in the significant growth of cloud computing, which has allowed businesses to become more intelligent and personalized (Drake, 2014; Fox, 2011). Cloud services encapsulate and integrate computing (hardware and software) resources and offer them on-demand as flexible and scalable web services to support businesses. Thus, an organization often achieves a significant increase in business capabilities and flexibility by moving its systems to the cloud. However, these benefits depend on having an optimal composition of multiple cloud services to support multiple related business processes. Therefore, it is very meaningful to research the optimal selection of cloud services, or the "cloud service composition optimal selection" (CSCOS) problem (Chen et al, 2019; Wu et al, 2020).

✉ Dongxiao Gu
gudongxiao@hfut.edu.cn

Extended author information available on the last page of the article

A typical organization has multiple related business processes, each consisting of multiple subtasks. For example, an organization might have an order-processing process, an inventory-control process, and a billing process. These processes are related to each other since processing an order will affect the inventory of the product and result in accounts receivable and the need to create a bill. Additionally, each process such as the order-processing process might have subtasks such as order verification, creation of a pick list, and packing. Multiple cloud services are generally available to support each subtask (Zheng et al, 2020). These cloud services have a variety of types and specifications and are offered by vendors such as Amazon, Google, and Microsoft with various Quality of Service (QoS) and price characteristics (Jain and Hazra, 2019; Nunez et al, 2021; Cohen et al, 2019; Bülbül et al, 2021), and thus the cloud services can be regarded as information technology products with certain characteristic parameters. Many times, organizations also build their own private cloud services to satisfy some specific requirements of the subtasks in terms of performance, and to address some native shortcomings of the public cloud, such as the security

risk posed by the multi-tenancy of the public cloud (Chen and Chang, 2020).

With the maturity of the cloud service product market and cloud computing technology, users can migrate between service providers to select cloud service products and optimize their composition (Rehman et al, 2015). Thus, an organization typically uses a hybrid cloud environment that consists of several types of cloud service products offered by multiple vendors as well as the organization's own private services, all working together to support multiple interrelated business processes (Passacantando et al, 2016). Effective design and management of this environment require organizations to select and compose an optimal set of cloud services to support the various subtasks of their business processes from a set of candidate cloud services (Li et al, 2016; Aghamohammadzadeh and Valilai, 2020).

This paper focuses on addressing the above challenges and proposes the multi-factor CSCOS model, the complexities of which lie in the need to address the following areas:

- From a process perspective, since multiple related processes consisting of subtasks support critical business operations, the selected services for each subtask in the process should achieve a holistic coordinated optimization (Mo et al, 2020; Scheepers and Scheepers, 2008).
- From a subtask perspective, some "special requirement" business tasks, like Internet-of-Things (IoT) or event-driven applications, tend to have a higher demand for cloud service performance (Adhikari et al., 2019), requiring a new deployment paradigm such as No Operations (NoOps) or serverless computing (Naseri and Navimipour, 2019).
- From a product perspective, various cloud service products are offered by different providers, and compatibility between selected products is one of the most important considerations in the adoption of a cloud service (Rehman et al, 2015; Yoo and Kim, 2018).
- From a performance perspective, the selected services should meet the QoS performance requirements of each subtask and the whole process, and should also achieve a better overall multi-process QoS performance (Khanouche et al, 2019).

To the best of our knowledge, none of the existing models for choosing a composition of cloud services comprehensively address the above complexities. The rest of this paper is structured as follows: The related works are summarized in Section 2. Section 3 formulates the multi-factor CSCOS model, and Section 4 presents the solution algorithm. Section 5 describes the process of generating a synthetic data set and numerical experiments to analyze the model and algorithms proposed in this paper. Section 6 concludes the paper and provides directions for future research.

2 Literature Review

Previous CSCOS research has considered various QoS characteristics for the selection of an optimal combination of web services to support the business process (Liang and Du, 2017; Yang et al, 2019). However, the maturity of the cloud services market has resulted in the availability of many services with a broad range of QoS characteristics from various vendors. This abundance of available services and the existence of multiple related business processes has made the service selection and composition problem more complex, requiring further research.

Most previous CSCOS research used an abstract representation of cloud services in terms of their QoS and tried to optimize QoS performance. A series of studies by Zhou and Yao considered cloud services as a collection of performance measures for a range of QoS values to pursue optimal QoS performance with different heuristic algorithms (Zhou and Yao, 2017b; Zhou et al, 2018). Gabrel et al. constructed the expression semantics of services based on the QoS values of each service and studied the service composition based on automatic semantics (Gabrel et al, 2018). Asghari and Navimipour studied the load balancing in cloud composition and used the inverted ant colony algorithm to optimize QoS in terms of time and cost (Asghari and Navimipour, 2019). In this paper, we not only optimize traditional QoS but also expand the connotation of cloud services by considering various types and brands of services, and thus enrich the optimization goals of CSCOS. The enriched goals include increasing the degree to which the special requirements of the process are met and the compatibility of the selected services in CSCOS.

Additionally, most previous CSCOS studies focused on a single process. Yang et al. discussed the large-scale CSCOS problem caused by the rapid increase in the number of candidate cloud services in one single cloud service composition process (Yang et al, 2019). Zhou and Yao developed a hybrid artificial bee colony combined with Archimedean copula estimation of the distribution algorithm to improve the search capability based on QoS (Zhou and Yao, 2017b). In this paper, we address the CSCOS problem in multiple related process scenarios and considers the coordination requirements between processes, to closely represent the real business environment supported by hybrid cloud services. To the best of our knowledge, none of the existing models comprehensively address this complex real-life situation.

Furthermore, a cloud service empowers business process management (BPM), which focuses on deploying the cloud service to support real business process scenarios to realize elastic processes. Schulte et al. did a comprehensive study on elastic BPM in the cloud, which conceptualized its architecture, reviewed existing works, and presented open issues and research directions (Schulte et al, 2015). Hoenisch et al. stud-

ied a novel class of BPM system (BPMS) called eBPMS and designed an elastic scheduling approach of eBPMSs that can also reduce the cost of hybrid cloud environments (Hoenisch et al, 2015). Furthermore, they addressed the service instance placement problem using an optimization model to realize the elastic business scheduling process under cost-effective optimization (Hoenisch et al, 2016). Waibel et al. focused on the fluctuating requirements for the computing resources of cloud-based BPMSs during the execution of business process activities; they introduced an elastic BPMS, the Vienna Platform for Elastic Processes on Containers (ViePEP-C), and presented a task scheduling algorithm based on ViePEP-C (Waibel et al, 2021). Euting et al. presented a method based on process knowledge for BPM-aware cloud computing to improve resource scaling decisions, and proposed an Infrastructure as a Service (IaaS) resource controller using fuzzy theory to monitor the resource requirements of process tasks (Euting et al, 2014). Janiesch et al. studied the opportunities and challenges of implementing a BPM-aware cloud architecture to optimize process runtime, and for the automation of task processes, which is the key to affecting the running time. They proposed a cloud-aware business process optimization method that provides computing resources based on process knowledge (Janiesch et al, 2014). Thus, recent BPM studies have focused on the technical details of cloud-based BPM to improve the efficiency and quality of cloud computing execution for BPM. They do not address the problem of selecting appropriate cloud services for multiple, related business processes. We focus on filling this gap.

Some previous research has considered the consistency between selected cloud services. Jin et al. considered QoS and correlation in cloud service composition and proposed a correlation description model to solve the QoS dependencies of services (Jin et al, 2017). Deng et al. proposed a correlation-aware service pruning method to solve the associated cloud service composition problem from a time and cost perspective (Deng et al, 2016). In this paper, we consider the overall consistency of CSCOS from the perspective of the brand of candidate cloud service. This is because services of the same brand will have better compatibility (Liang et al, 2021).

Since CSCOS has NP-hard complexity, heuristic evolutionary algorithms have been the main solution approach. Qi et al. developed a knowledge-based differential evolution (DE) algorithm for CSCOS and used simulated data to prove that it performs better than previous algorithms (Qi et al, 2018). Zhang et al. established a fuzzy multi-objective linear programming model of CSCOS in the transportation field and used a genetic algorithm (GA) algorithm to solve it (Zhang et al, 2018). Yang et al. studied large-scale CSCOS and proposed a dynamic GA combined with the ant colony algorithm to guarantee good solution accuracy and stability (Yang et al, 2019). Naseri and Navimipour proposed an agent-based particle swarm optimization (PSO) solution for QoS-aware CSCOS (Naseri and Navimipour, 2019). This paper proposes the C3IMDE algorithm to solve multi-factor CSCOS and compares the results with DE, GA, and PSO approaches as the benchmark.

3 Formulation and Solution of Multi-Factor CSCOS Model

3.1 Problem Description

Let WP represents a set of m related business processes of an organization; each process is denoted as $P_i, i = 1, \cdots, m$. In the i^{th} process, ST_{ij} denotes the j^{th} subtask, $j = 1, \cdots, n_i$. The set of candidate cloud services available for subtask ST_{ij} is represented as $CCSS_{ij}$. In the $CCSS_{ij}$, each candidate cloud service is denoted as S_{ijk}, $k = 1, \cdots, l_{ij}$, and $S_{ijk} \in CCSS_{ij}$. Figure 1 shows the CSCOS problem of an organization that is addressed in this paper.

The top of Fig. 1 shows a number of processes of the organization with their subtasks. These subtasks include general subtasks and subtasks with special requirements. The desired objectives are at the right.

The bottom of Fig. 1 shows all the possible cloud services available for each subtask. Different colors represent different brands, while different shapes represent types of cloud service.

The middle part of Fig. 1 shows the scenario of selecting cloud services for each subtask in process p_i. An organization needs to decide which cloud service S_{ijk} to select for subtask ST_{ij} from its candidates $CCSS_{ij}$. Let the subtask decision variable on ST_{ij} be denoted by s_{ij}, so that $s_{ij} \in \{1, \cdots, l_{ij}\}$. Thus, all subtask decision variables of ST_{ij} form the process decision variable of p_i, which is a vector $(s_{i1}, s_{i2}, \cdots, s_{in_i})$. For example, in Fig. 1, the process decision variable of p_i is identified as $(6, 3, 1, 3, \cdots, 7)$ where the number represents the serial number of services in the candidate set. Therefore, the CSCOS problem can be defined as the problem of finding the optimal decision $(s^*_{i1}, s^*_{i2}, \cdots, s^*_{in_i})$ for each process of an organization. The decision considers the factors listed in the following section.

Table 1 describes the types of services offered, as shown in the bottom of Fig. 1, with popular brands listed as examples.

3.2 QoS Attributes

Cloud service products reflect service quality through their QoS performance. In order to measure the QoS performance, the QoS attributes of cloud services have been divided into multiple dimensions, and each dimension is measured by multiple QoS characteristics; for example, dimen-

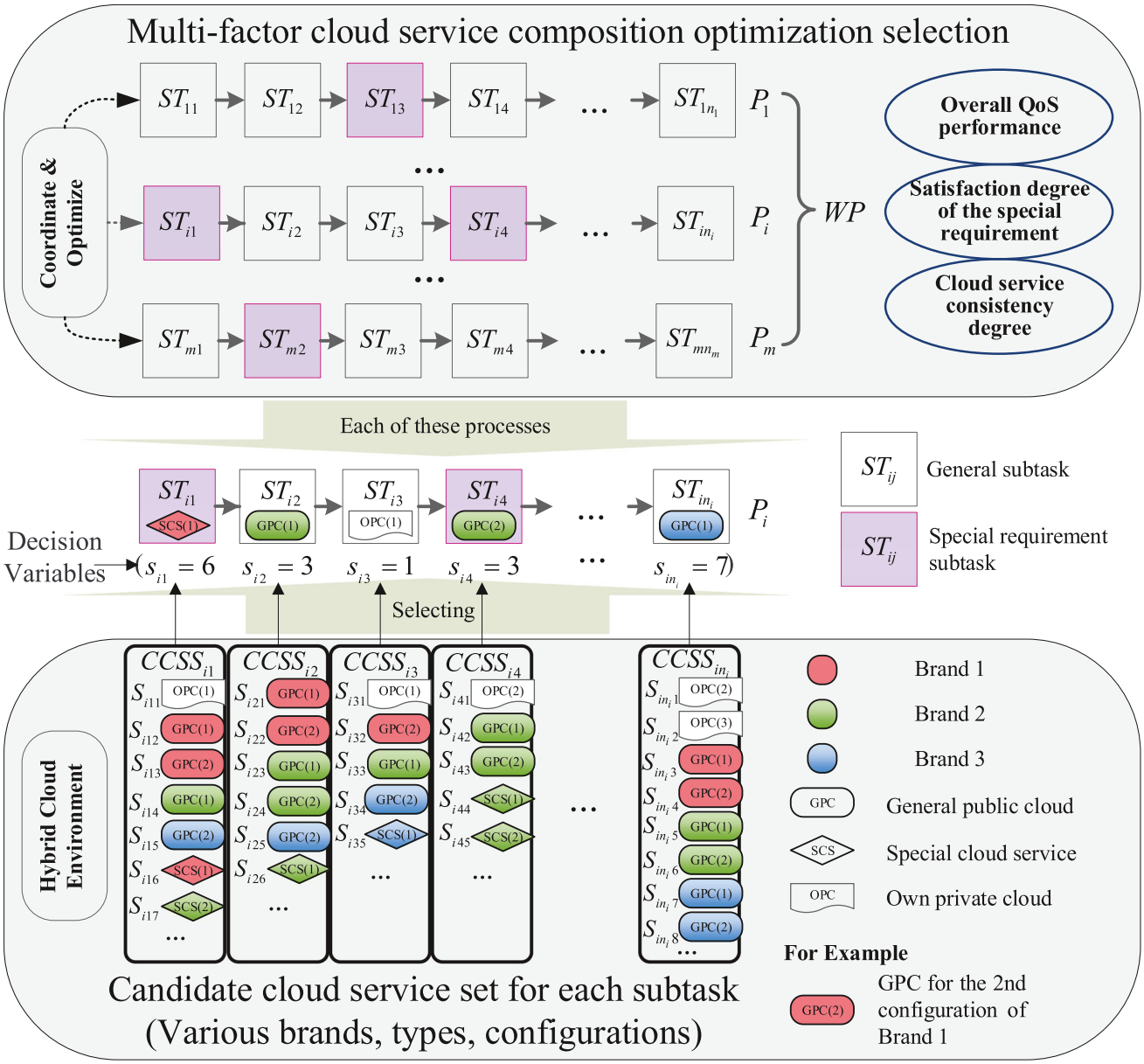

Fig. 1 The CSCOS problem of an organization in a hybrid cloud environment

sions include the service dimension (measurement includes response time, reliability, availability, and successful execution rate), the product dimension (measurement includes product maturity, product price, and cost-effectiveness), and

Table 1 Service types in the hybrid cloud environment

Type	Symbol	Example brands	Characteristics and description
General public cloud	GPC	Amazon AWS EC2, Microsoft Windows Azure, Google cloud	General public cloud service provided as a web service, selected for a single subtask in the business processes.
Special cloud service	SCS	AWS Lambda, Alibaba Function Compute	Special cloud service that has a special function, e.g., providing a NoOps serverless framework.
Own private cloud	OPC	Organization's own cloud services	Private cloud service built by the organization, using private IT equipment.

the enterprise dimension (measurement includes enterprise reputation and capability) (Xue et al, 2016). Previous studies on cloud service composition selected a subset of QoS characteristics as the main characteristics to measure QoS. For example, Yang et al. considered the generalizability of the research and select time (the execution duration of task processing), cost (the total cost paid by the user during the entire task execution process), availability (the ability to invoke the cloud service during the time period), and reliability (the ability to successfully perform tasks within a given time and condition) to evaluate the QoS. The values of these characteristics are generally available from service providers as the specifications of cloud service products (Yang et al, 2019). Based on the previous studies summarized in Table 2, four QoS characteristics (availability, reliability, time, cost) have been selected first to measure the QoS performance of cloud services in this paper.

In addition to the above characteristics, the trust of users in the cloud service vendor should also be considered. Thakur and Breslin believed that the reputation of the cloud provider and their products can be used in service selection, as it helps a user to choose an appropriate cloud provider and cloud service that will increase the user's trust (Thakur and Breslin, 2019). Reputation and trust are often used as a QoS characteristic to measure the QoS performance of cloud service products (Xue et al, 2016). Thus, reputation has been added to the QoS characteristic set in this paper. Therefore, five QoS characteristics (availability, reliability, time, cost, and reputation) represent the specifications of cloud service products. Table 3 provides the definition of these QoS characteristics.

Since in real life it is not practical to change cloud services dynamically, the values of QoS characteristics used represent a static performance of the services. The static values of the five QoS characteristics are a macro abstraction mean performance in most operating situations, and do not represent the dynamic actual value of the QoS during the specific execution of cloud services (Zhou and Yao, 2017a, b, c;

Table 3 Description of QoS characteristics

QoS characteristic	Description
Availability	Accessibility specification; the average ratio of successful invocation during the service execution; represented by the symbol ava, the value type is a percentage, $ava \in [0, 1]$.
Reliability	Stability specification; the average ratio of successfully performing tasks (successfully completed tasks to total tasks) during the service execution; represented by the symbol rel, the value type is a percentage, $rel \in [0, 1]$.
Time	Efficiency specification; the total service duration time to perform a unit task volume, including execution time and response time; represented by the symbol tim, the value type is a real number, $tim \in (0, +\infty)$.
Cost	Price specification; the total cost of service to the user, which represents the price of the service; represented by the symbol cos, the value type is a real number, $cos \in (0, +\infty)$.
Reputation	Trust specification; the level of trust the user has in the cloud service provider and their various products, which usually reflects the cloud service's trustworthiness based on the impressions of users of past products and word of mouth; represented by the symbol rep, the value type is an integer number, $rep \in \{1, 2, \ldots, 10\}$.

Table 2 Summary of QoS characteristics

Literature reference	QoS characteristics explored
Tao et al (2009)	Categorized QoS characteristics as the performance QoS and the description QoS. The former includes time, reliability, maintainability, and cost. The latter includes trust.
Khurana et al (2016)	Surveyed the QoS characteristics, including availability, reliability, scalability, reusability, elasticity, adaptability, security, cost, efficiency, response time, throughput, and so on, and found the top ranked characteristics and their percentage use in various frameworks as response time (14%), availability (10%), security (10%), cost (10%), performance (10%), and reliability (8%).
Xue et al (2016)	Studied the service dimension (time, reliability, availability, and successful execution rate), the product dimension (maturity, price, and cost-effectiveness), and the enterprise dimension (reputation and capability).
Ding et al (2017)	Divided QoS into benefit QoS (the forward QoS) and cost QoS (the backward QoS) and selected time and throughput as two characteristics.
Zhou and Yao (2017a)	Discussed the reputation of cloud service providers, which reflects users' trust in the cloud service products, and finally chose price, time, availability, and reputation as measures of QoS performance.
Yang et al (2019); Zhou and Yao (2017b)	Considered time, cost, availability, and reliability to establish a QoS evaluation model.
Zhou and Yao (2017c)	Used four QoS attributes, i.e., time, price, reliability, and reputation to establish a QoS evaluation model.
Thakur and Breslin (2019)	Studied the effect of users' trust on cloud providers, taking the reputation of cloud services as an important measure for forming a cloud federation.

Yang et al, 2019; Jin et al, 2017). In addition, there are some correlations between these QoS characteristics. For example, high-quality cloud services tend to have high availability and reliability, or the values of QoS characteristics are also related to the type and brand of the cloud service. We handle these correlations from a static perspective in a synthetic data generation process discussed in Section 5.1. From a dynamic perspective, during the execution of cloud services, the QoS characteristics will change dynamically based on operating conditions. However, the average performance of the service over a period should be close to the values used in service selection. The modeling of dynamic performance is outside the scope of this paper.

3.3 Multi-factor CSCOS Model

To express the model more clearly, we summarize the notations used in the model as model parameters in Table 4 and those used as intermediate values in Table 5.

3.3.1 Optimization Objective

QoS performance *The QoS performance of subtasks:* let $q_{c,S_{ijk}}$, $c \in \{ava, rel, tim, cos, rep\}$ describe the values of the five QoS characteristics of candidate cloud service S_{ijk}. As mentioned above, the larger the value of $q_{c^+,S_{ijk}}$, $c^+ \in \{ava, rel, rep\}$, the better the quality of the cloud service and higher preference of users; c^+ is the forward QoS characteristics. By contrast, the smaller the value of $q_{c^-,S_{ijk}}$, $c^- \in \{tim, cos\}$, the shorter service duration time of the cloud service and the lower cost, and the higher the preference of users; c^- is the backward QoS characteristics (Ding et al, 2017; Yang et al, 2019). In addition, each QoS characteristic has different units of measure and scale: availability and reliability are represented as a percentage, time and cost are real numbers, and reputation refers to the ordered sequence of levels. In order to obtain the QoS performance for subtasks, Eqs. 1 and 2 are used to normalize the value of each QoS characteristic to the scale [0, 1], and to unify the forward and backward QoS characteristics as forward QoS characteristics (Ding et al, 2017; Xue et al, 2016; Yang et al, 2019; Zhou and Yao, 2017a). This normalization and unification allow us to integrate these QoS characteristics and avoid the complexities caused by having different preferred directions and units of QoS characteristics.

$$nq_{c^+,ST_{ij}} = \begin{cases} \frac{q_{c^+,S_{ijk}} - q^{min}_{c^+,CCSS_{ij}}}{q^{max}_{c^+,CCSS_{ij}} - q^{min}_{c^+,CCSS_{ij}}} & , \text{if } (q^{max}_{c^+,CCSS_{ij}}) \neq (q^{min}_{c^+,CCSS_{ij}}) \\ 1 & , \text{if } (q^{max}_{c^+,CCSS_{ij}}) = (q^{min}_{c^+,CCSS_{ij}}) \end{cases}. \quad (1)$$

$$nq_{c^-,ST_{ij}} = \begin{cases} \frac{q^{max}_{c^-,CCSS_{ij}} - q_{c^-,S_{ijk}}}{q^{max}_{c^-,CCSS_{ij}} - q^{min}_{c^-,CCSS_{ij}}} & , \text{if } (q^{max}_{c^-,CCSS_{ij}}) \neq (q^{min}_{c^-,CCSS_{ij}}) \\ 1 & , \text{if } (q^{max}_{c^-,CCSS_{ij}}) = (q^{min}_{c^-,CCSS_{ij}}) \end{cases}. \quad (2)$$

In these equations, $q_{c^+,S_{ijk}}$ and $q_{c^-,S_{ijk}}$ are the corresponding QoS characteristic values of S_{ijk}, and $nq_{c^+,ST_{ij}}$ and $nq_{c^-,ST_{ij}}$ are the normalized QoS performances of the corresponding characteristic on the subtask ST_{ij} that selects S_{ijk}. $q^{max}_{c^+,CCSS_{ij}}$, $q^{min}_{c^+,CCSS_{ij}}$, $q^{max}_{c^-,CCSS_{ij}}$, and $q^{min}_{c^-,CCSS_{ij}}$ are the maximum and minimum value of the corresponding QoS characteristics of all candidate cloud services in $CCSS_{ij}$. When c is c^+ or c^-, the QoS performance of subtasks $nq_{c,ST_{ij}}$ is also determined as the corresponding one, of $nq_{c^+,ST_{ij}}$ and $nq_{c^-,ST_{ij}}$.

The QoS performance of processes: The process flow can have various patterns such as sequential, splits, loop, and so on. However, previous literature studies suggest that most commonly used process patterns can be converted to sequential patterns (Xue et al, 2016; Yang et al, 2019; Zhou and Yao, 2017b). Thus, to reduce the model complexity, this paper assumes the process is composed of subtasks arranged in a sequential pattern. Therefore, the evaluation of the QoS characteristics of business processes at a macro level can be done by assuming that subtasks of a business process form a simple sequential pattern (the workflow completes sequentially on the cloud service for these subtasks in sequence). Thus, the QoS performance of process

Table 4 Model parameters

Variables	Definition and description
$q_{c^+,S_{ijk}}$; $q_{c^-,S_{ijk}}$	The corresponding QoS characteristic specification value of cloud service product S_{ijk}, the forward QoS characteristics $c^+ \in \{ava, rel, rep\}$ and the backward QoS characteristics $c^- \in \{tim, cos\}$.
$\widetilde{nq}_{c,ST_{ij}}$; $\widetilde{nq}_{c,P_i}$	The normalized threshold of QoS feature $c \in \{ava, rel, tim, cos, rep\}$ on subtask ST_{ij} and process P_i, which the result of CSCOS needs to meet.
$SRST_{P_i}$	The set of subtasks with special requirements in the process P_i, $\lvert SRST_{P_i} \rvert$ is the quantity of elements of this set.
ω_c	The weight of each QoS feature c, which is used to integrate and derive overall QoS performance.
ω_1; ω_2; ω_3	The weights of the overall QoS performance ($TQoS$), the overall satisfied special requirement score (TSR), and the overall cloud service consistency score (TSC) of the multi-process set, which are used to derive the score of the multi-factor CSCOS model.

Table 5 Intermediate values

Variables	Definition and description
$TQoS$; TSR; TSC	The overall QoS performance, the satisfied special requirement score, and the cloud service consistency score of the multi-process set, respectively.
$nq_{c^+,ST_{ij}}$; $nq_{c^-,ST_{ij}}$	The normalized QoS performance of corresponding QoS characteristics c^+ and c^- of the subtask ST_{ij} that selects S_{ijk}.
$nq_{c,ST_{ij}}$; nq_{c,P_i}; $nq_{c,WP}$	The normalized QoS performance of corresponding QoS characteristic c of the subtask ST_{ij}, of the process P_i, and of the multiple processes, respectively, when S_{ijk} has been selected on the ST_{ij}.
$n_{P_i,l}$	The special requirement scores of the l^{th} special requirement subtask in the process P_i, 0 means not satisfied, 1 means satisfied.
SCF_{WP}	The $Node$ consistency of the multi-process set.
SCF_{AEP}	The average $Node$ and $Side$ consistency of all processes.
$EoBN_{WP}$	The entropy of $Node$ type of the multi-process set. $EoBN_{WP}^{max}$ is the maximum entropy under the most chaotic situation of $Node$ type.
$P_{WP,TN}$	The proportion of the number of the same $Node$ type to the total number of subtasks in the multi-process set.
SCF_{P_i}	The consistency of the process P_i.
$CEoBN_{P_i}$; $CEoBS_{P_i}$	The continuous cumulative effect modified entropy of $Node$ type and $Side$ type of the process P_i, respectively. $CEoBN_{P_i}^{max}$ and $CEoBS_{P_i}^{max}$ is the maximum value under the most chaotic situation of $Node$ type and $Side$ type of the process P_i, respectively.
$cue_{P_i,CoTN}$; $cue_{P_i,CoTS}$	The continuous cumulative effect of $Node$ type and $Side$ type, respectively.
$P_{P_i,TN}$; $P_{P_i,TS}$	The proportion of the number of the same $Node$ type and $Side$ type to the total number of $Node$ and $Side$ in the process P_i, respectively.
$P_{P_i,CoTN}$; $P_{P_i,CoTS}$	The proportion of the number of the same type of $Node$ and $Side$, which are an aggregated block to the total number of $Node$ and $Side$ in the process P_i, respectively.

nq_{c,P_i} can be derived by aggregating the QoS performance of all subtasks in this process, which are expressed as simple macro measures (Zhou and Yao, 2017a, b, c; Qi et al, 2018; Khanouche et al, 2019; Yang et al, 2019; Xue et al, 2016): $nq_{ava,P_i} = \prod_j nq_{ava,ST_{ij}}$, $nq_{rel,P_i} = \prod_j nq_{rel,ST_{ij}}$, $nq_{tim,P_i} = \sum_j nq_{tim,ST_{ij}}$, $nq_{cos,P_i} = \sum_j nq_{cos,ST_{ij}}$, and $nq_{rep,P_i} = (\sum_j nq_{rep,ST_{ij}})/n_i$.

The QoS performance of the multiple processes: The performance of each characteristic c of the multiple processes ($nq_{c,WP}$) is the average of the performance corresponding to this characteristic in each process P_i, $nq_{c,WP} = (\sum_{i=1}^{m} nq_{c,P_i})/m$. The QoS performance of the multiple processes, $TQoS$, is calculated as the weighted sum of the five QoS characteristics:

$$TQoS = \sum\nolimits_c \omega_c nq_{c,WP}. \tag{3}$$

Special requirements of subtasks Some subtasks may have special requirements that need to be met by the selected service. For example, a subtask may need to adopt a serverless framework to pursue NoOps for high efficiency.

Special requirement subtask (SRST) is subtask in the process that require specific needs to be met. In this paper, meeting a special requirement refers to selecting a special cloud service product (SCS-type service) for the subtask that meets the special requirement. The special requirements are not constraints, meaning that although meeting them is an objective, doing so is not absolutely required, since SCS-type services tend to have high costs; for example, serverless computing is also viewed as an economically driven service (Patros et al, 2021).

We calculate the satisfaction degree of the special requirement of all SRSTs in the process P_i as $(\sum_{l=1}^{|SRST_{P_i}|} n_{P_i,l})/|SRST_{P_i}|$. Where $SRST_{P_i}$ is the SRST set in the process P_i. $n_{P_i,l}$ is the special requirement score of the l^{th} SRST in the process P_i. If a SCS-type service is selected for the l^{th} SRST in the process, then $n_{P_i,l} = 1$. Otherwise, the special requirement is not satisfied, and $n_{P_i,l} = 0$. TSR is the satisfaction degree of the special requirement of overall multiple processes, which is the average of the satisfaction degree of the special requirement of all processes.

$$TSR = \left(\sum\nolimits_{i=1}^{m} (\sum\nolimits_{l=1}^{|SRST_{P_i}|} n_{P_i,l})/|SRST_{P_i}|\right)/m. \tag{4}$$

Compatibility between selected cloud services The selected cloud services need to be compatible so they can work together to execute the process successfully and minimize composition efforts (Yoo and Kim, 2018; Xu et al, 2019; Gu et al, 2019). It is preferable to choose the same brand of service, provided by the same vendor, because this choice

brings low maintenance costs and possibly better compatibility (Deng et al, 2016; Liang et al, 2021).

The vendors offering services are represented by the brand. The compatibility between selected cloud services has two aspects: the cloud service brand selected for each subtask in the process, represented as $Node$, and the brands of cloud services selected for two connected subtasks in the process, represented as $Side$. The former reflects the consistency of information processing technology, and the latter reflects the ease of information transfer between two adjacent subtasks.

Figure 2 shows six example scenarios labeled (a) through (f) for a process with six subtasks. The subscript of $Node$ and $Side$ represents the type. The $Node$ type represents different brands, and the $Side$ type represents the connection between different types of $Nodes$. The superscript of $Node$ and $Side$ is the serial number of the same type of $Node$ and $Side$ in the process from left to right.

To explain more clearly, Fig. 2(a) and (f) are enlarged with additional detail in Fig. 3(a) and (f). In Fig. 3(a), all $Node$ and $Side$ are of the same type ($Node_1$ and $Side_1$). The superscripts of $Node_1$ and $Side_1$ have values from 1 to 6 and from 1 to 5, respectively. Figure 3(f) has three types of $Node$ ($Node_1$, $Node_2$, $Node_3$) and three types of $Side$ ($Side_4$, $Side_5$, $Side_6$). The superscript of the green $Node$ type 2 ($Node_2$) has values 1, 2, 3 from left to right. The superscript of $Side$ type 4 ($Side_4$) has values 1, 2, 3 from left to right, and $Side$ types 5 and 6 have the superscript value of 1. Based on the above symbol description, the six example scenarios in Fig. 2(a) through (f) can be compared for consistency, as described below.

From the $Node$ consistency perspective, Fig. 2(a) is better than (b) since in the former, all subtasks select one brand of service, whereas the latter contains two brands with an equal number of services. In other words, the fewer the types of $Node$, the better the consistency of the process. In Fig. 2(b) and (c), there are equal numbers of $Node_1$ and $Node_2$, but the former is better than the latter, since in Fig. 2(b) the same type of $Node$ is connected together. In Fig. 2(d), (e), and (f), the $Node$ consistency is the same, since they have the same type of $Node$ with an equal number and a similar aggregation situation. However, they have different connection situations of $Node$ that lead to different consistencies of $Side$.

From the $Side$ consistency perspective, comparing Fig. 2(d) and (e), there are two $Side$ types in Fig. 2(d), whereas Fig. 2(e) has three $Side$ types, so Fig. 2(d) is better. In other words, the fewer the type of $Side$, the better the consistency of the process. Comparing Fig. 2(e) with (f), both have the same number of $Node$ and $Side$ types, but Fig. 2(e) is better because three $Side_4$ are aggregated together ($Side_4^1$, $Side_4^2$, $Side_4^3$ are next to one another), so that aggregated blocks of the same type of $Side$ can be formed in the process.

In conclusion, the cloud service consistency of a process can be measured by two factors: (1) first, the degree of chaos

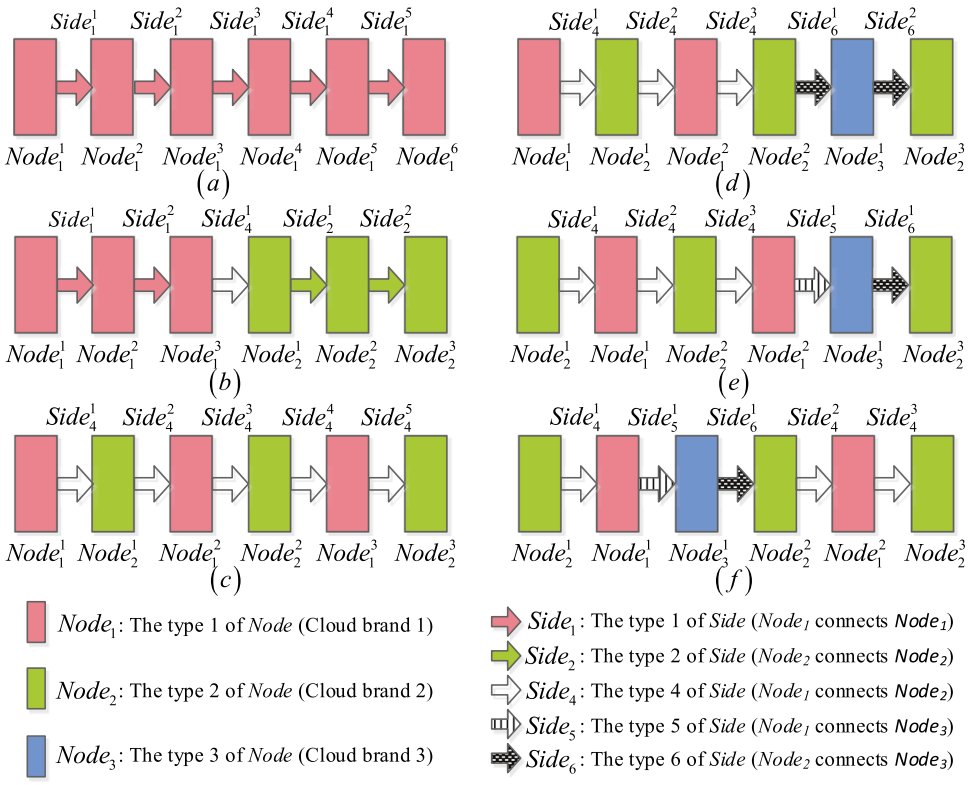

Fig. 2 A representation of cloud service consistency

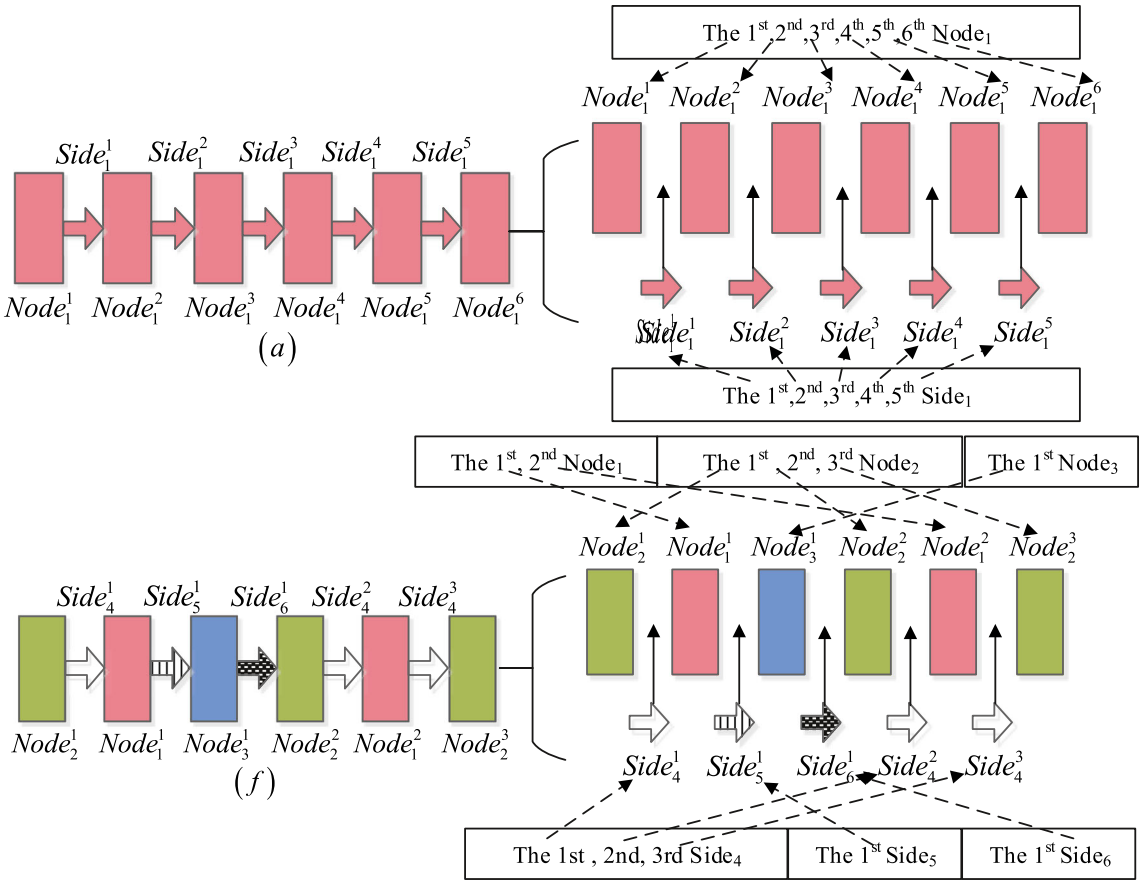

Fig. 3 Enlarged figures show the detail of connection

of the types of $Node$ and $Side$ in a process, which can be measured by entropy, and (2) second, the aggregation effect of the same type of $Node$ and $Side$, which is reflected by the continuous cumulative effect. Thus, the consistency of a process is measured by the entropy method modified by the continuous cumulative effect.

In addition to considering consistency in a process above, it is necessary to measure the consistency of the multi-process set. For example, imagine if two processes have the same number of subtasks, but one process selects brand one for all subtasks, and the other selects brand two for all subtasks. The processes have the same consistency based on the above single-process perspective, but overall, this situation is not as good as the consistency of both processes selecting the same brand for all subtasks. Thus, for the multi-process set, consistency can be measured by the chaos of $Node$, which can be calculated by the entropy of $Node$. Since each process of the multiple processes is separate, the connecting $Side$ between two processes does not play a role. Thus, the $Side$ consistency is not considered in the multi-process set in this paper. Similarly, the aggregation effects of $Node$ and $Side$ are also not considered in the multi-process set.

Based on the above information, the overall cloud service consistency of multiple processes, TSC, is measured by the balanced mean of $Node$ consistency of the multi-process set SCF_{WP} and the average $Node$ and $Side$ consistency of all processes SCF_{AEP}:

$$TSC = (SCF_{WP} + SCF_{AEP})/2. \tag{5}$$

The normalized $Node$ consistency of the multi-process set is calculated as:

$$SCF_{WP} = (EoBN_{WP}^{max} - EoBN_{WP})/(EoBN_{WP}^{max}). \tag{6}$$

where, the entropy of the $Node$ type of the multi-process set, $EoBN$, is

$$EoBN_{WP} = -\sum\nolimits_{TN} P_{WP,TN} \ln(P_{WP,TN}). \tag{7}$$

$EoBN_{WP}^{max}$ represents the most chaotic situation of $Node$ type in the multi-process set, where each subtask has selected a different brand of cloud service. $P_{WP,TN}$ is the proportion of the number of the same $Node$ type to the total number of

subtasks in the multi-process set. TN is the 'type' label of $Node$.

The average $Node$ and $Side$ consistency of all processes is as follows:

$$SCF_{AEP} = \left(\sum_{i=1}^{m} SCF_{P_i}\right)/m. \tag{8}$$

The consistency of the process P_i is the balanced mean of $Node$ and $Side$ consistency:

$$SCF_{P_i} = \left(\frac{CEoBN_{P_i}^{max} - CEoBN_{P_i}}{CEoBN_{P_i}^{max}} + \frac{CEoBS_{P_i}^{max} - CEoBS_{P_i}}{CEoBS_{P_i}^{max}}\right)/2. \tag{9}$$

The continuous cumulative effect modified entropy of $Node$ types and $Side$ types of the process P_i can be represented as follows:

$$\begin{cases} CEoBN_{P_i} = -\sum_{TN}(1-\sum_{CoTN} cue_{P_i,CoTN}) P_{P_i,TN} \ln(P_{P_i,TN}) \\ CEoBS_{P_i} = -\sum_{TS}(1-\sum_{CoTS} cue_{P_i,CoTS}) P_{P_i,TS} \ln(P_{P_i,TS}) \end{cases}. \tag{10}$$

$CEoBN_{P_i}^{max}$ and $CEoBS_{P_i}^{max}$ are the most chaotic situations of the types of $Node$ and $Side$ in the process P_i, where each $Node$ and $Side$ type is different. TN and TS are the type labels of $Node$ and $Side$, and $CoTN$ and $CoTS$ are the labels of aggregated blocks of the same type of $Node$ and $Side$ by TN and TS in the process P_i, respectively. (The same type of $Node$ and $Side$ may have multiple aggregated blocks in the process P_i). The continuous cumulative effect of the $Node$ and $Side$ type is calculated as follows:

$$\begin{cases} cue_{P_i,CoTN} = \left(e^{P_{P_i,CoTN}\ln 2} - 1\right)^{\lambda_{Node}} \\ cue_{P_i,CoTS} = \left(e^{P_{P_i,CoTS}\ln 2} - 1\right)^{\lambda_{Side}} \end{cases}. \tag{11}$$

$P_{P_i,CoTN}$ and $P_{P_i,CoTS}$ are the proportion of the number of the same type of $Node$ and $Side$ in the $CoTN$ and $CoTS$ aggregated block to the total number of $Node$ and $Side$ in the process P_i, respectively. The continuous cumulative effect, $cue_{P_i,CoTN}$ and $cue_{P_i,CoTS}$, emphasizes the degree of aggregated togetherness of the same $Node$ and $Side$ type. The parameters λ_{Node} and λ_{Side} are the extent factors, which are used to expand and reduce the extent of the continuous cumulative effect.

3.3.2 Constraint Condition

Since some subtasks in the process may have the minimum QoS requirement for the services, the selected cloud service for these subtasks should meet the minimum thresholds, which are specified by the users based on business needs. We standardize the threshold values using Eqs. 1 and 2 to get the constraints $\widetilde{nq}_{c,ST_{ij}}$ that must be satisfied by the selected cloud service for each subtask ST_{ij} that has the minimum QoS requirement. Therefore, a model constraint is expressed as:

$$nq_{c,ST_{ij}} \geq \widetilde{nq}_{c,ST_{ij}}. \tag{12}$$

Candidate cloud services for each subtask, which need to meet the inequality in Eq. 12, can be reserved for the subsequent optimization algorithm for selecting services; other candidate cloud services that do not meet the minimum requirements of the subtasks will not be considered for deployment on those subtasks.

Furthermore, even though each subtask constraint in a process is satisfied, the whole process may need to satisfy a higher process-level QoS constraint, which is better by a factor α_{c,P_i} of the aggregation of each subtask's minimum required QoS. For example, availability is $\widetilde{nq}_{ava,P_i} = (1+\alpha_{c,P_i})\prod_j \widetilde{nq}_{ava,ST_{ij}}$. Thus, for process P_i the QoS constraint is as follows:

$$nq_{c,P_i} \geq \widetilde{nq}_{c,P_i}. \tag{13}$$

In addition to the above constraints, there is an implicit constraint that the decision variables are selected in accordance with common sense. Based on the problem description in Section 3.1, the subtask decision variable for ST_{ij} is s_{ij}, which is an integer variable representing the serial number k of cloud service S_{ijk}, which is selected for corresponding subtask ST_{ij} from its candidate cloud services set $CCSS_{ij}$. Thus, for the cloud service $S_{ijk}, k = 1, \cdots, l_{ij}$ in $CCSS_{ij}$, the implicit common-sense constraint to be satisfied by the subtask decision variables is,

$$s_{ij} \in \{1, \cdots, l_{ij}\}. \tag{14}$$

3.3.3 Overall Model of the Multi-Factor CSCOS

Based on the above description, the multi-factor CSCOS is to optimize the overall QoS performance, the average degree of the subtask special requirements that are satisfied, the compatibility of selected services, and the need for multi-process coordination. The objective function of multi-factor CSCOS contains three parts: $TQoS$, TSR, and TSC. Since these three parts have normalized values between 0 and 1, the total score of the objectives can be computed as the weighted linear combination of them. Weights can be assigned to each of these components by the users of the model based on the importance of each in the problem situation. The constraints of the model aim to satisfy the needs of each subtask and process. Thus, we formulate the multi-factor CSCOS as a constrained combinatorial optimization model:

$$\begin{gathered} \text{Max } Score = \omega_1 TQoS + \omega_2 TSR + \omega_3 TSC. \\ \text{Subject To } nq_{c,ST_{ij}} \geq \widetilde{nq}_{c,ST_{ij}}, nq_{c,P_i} \geq \widetilde{nq}_{c,P_i}. \\ i = 1, \cdots, m;\ j = 1, \cdots, n_i;\ s_{ij} \in \{1, \cdots, l_{ij}\};\ c \in \{ava, rel, cos, tim, rep\}. \end{gathered}$$

The weight $\omega_1, \omega_2, \omega_3$ indicates the importance of each part assigned by the model user. As a decision maker, the organization can set different weights according to business requirements or preferences for the three aspects. When the organization needs to strengthen the CSCOS decision-making in a certain aspect, it can set a higher weight in that aspect. The impact of weight changes on the optimal solution is discussed in detail in Section 5.2.3.

4 Model Solution

An improved differential evolution (DE) based on a constructive cooperative coevolutionary framework (C3IMDE) algorithm is designed to solve the above optimization model. The constructive cooperative coevolutionary (C3) framework coordinates the optimization of multiple business processes. The improved DE (IMDE) algorithm does the inner optimization of each process. A filtering method, an adaptive penalty method, and the boundary-modifying design are used as a subtask QoS constraint, process QoS constraint, and the implicit common-sense constraint, respectively.

4.1 Evolutionary Algorithms for Model Solution

4.1.1 Constructive Cooperative Coevolutionary (C3) Framework

The C3 framework has been proposed to solve the collaborative optimization problem described above (Glorieux et al, 2017). It can coordinate multiple processes while optimizing each process, resulting in overall collaborative optimization. Specifically, every process called an aim process is optimized by an inner optimization algorithm, using the fitness value of the multi-process set solution that combines the solution of the optimized process and already existing solutions of the other processes, called the environment processes.

Algorithm 1 provides the pseudo code of C3.

Algorithm 1 Pseudo code of C3

Require: the candidate cloud service set CS_{ij}
Ensure: the Result solution set *ResultSet*
1: **while** the final termination condition of stage 1 of C3 is not true **do**
2: **for** the process i = 1 to m and $CEPSS_i = \emptyset$ **do**
3: $Subpop_i = Initi\ (CS_i)$
4: $eps_i = BestUnexp\left(CEPSS_{Notempty(1\to m-i)}\right)$
5: put $Aimp_i^{1\to k} = IMDE\left(Subpop_i, eps_i\right)$ into $CEPSS_i$
6: **end for**
7: $iwes = Combine(epss_{1\to m}^{best})$
8: Remove the explored $epss_{i=old}^{best}$
9: **while** the final termination condition of stage 2 of C3 is not true **do**
10: **for** each process i =1 to m **do**
11: **if** this is the first iteration of stage 2 **then**
12: $Subpop_i = Initi\ (CS_i)$
13: $eps_i = iwes_{1\to i-1,i+1\to m}$
14: Put $Aimp_i^{1\to k} = IMDE\left(Subpop_i, eps_i\right)$ into $CMPSS_i$
15: The best $wps = combine(Aims_i^{1\to k}, eps_i)$ create $ResultSet$
16: **else**
17: $Subpop_i = CMPSS_i$
18: $eps_i = BestUnexp\left(CMPSS_{Notempty(1\to m-i)}\right)$
19: $Aimp_i^{1\to k} = IMDE\left(Subpop_i, eps_i\right)$ update $CMPSS_i$
20: The best $wps = combine(Aims_i^{1\to k}, eps_i)$ update $ResultSet$
21: **end if**
22: **end for**
23: **end while**
24: **end while**

Stage 1: The first stage of C3 is to create the candidate environment process solution set (*CEPSS*) and to obtain an initial solution (*iwes*), which is depicted in Algorithm 1 (lines 2-6).

At the first iteration, *CEPSS* is empty, and the aim process (*Aimp*) is optimized in turn by the inner optimization algorithm IMDE. IMDE evolves the initial random population from the candidate cloud service set of *Aimp*, combines it with the environment processes solution (*eps*), and stores the solution set of *Aimp* in *CEPSS*. The *eps* combines the best unexplored solution of other processes (except *Aimp*) that have been optimized and stored in *CEPSS*. When all processes are optimized, an *iwes* can be obtained by combining the best solution of each process in *CEPSS* in each iteration. After an *iwes* is created, starting from the last process, the longest-used best solution is deleted from *CEPSS*. If the stored solution of a process in *CEPSS* is empty, Algorithm 1 (lines 2-6) is used to supplement it.

Stage 2: The second stage of C3 is to build the candidate multi-process solution set (*CMPSS*) and to obtain the result of the overall multi-process solution set (*ResultSet*), which is depicted in Algorithm 1 (lines 7-23).

The first iteration of the second stage of C3 is to build *CMPSS* by the IMDE optimization of each process in turn. This is like the creation of *CEPSS*, except that *eps* combines the solutions of other processes in *iwes* that are obtained by stage 1 of C3. The *ResultSet* is created by using IMDE to optimize all processes in turn. After *CMPSS* is built, the initial population of *Aimp* is the solution stored in the corresponding process in *CMPSS*, and *eps* combines the best unexplored solution of other processes from *CMPSS*. Optimizing the *Aimp* in turn allows the solutions of *Aimp* to be used to update *CMPSS*, and the solution of the multi-process set is stored in *ResultSet*. Finally, the best solution in *ResultSet* is the result of C3.

4.1.2 Improved Differential Evolution (IMDE)

The DE algorithm has a highly efficient search ability for the exploitation and exploration of the population (Opara et al, 2019; Li et al, 2017). We design multi-type aim genes to improve DE as the inner optimization algorithm to optimize each process. The aim gene is the process decision variable, and the subtask decision variable at each gene locus is encoded with an integer that represents the serial number of selected cloud service for corresponding subtask from its candidate cloud service. The IMDE has three parts, described in pseudo code in Algorithm 2.

Algorithm 2 Pseudo code of IMDE

Require: the initial population *IP* of *Aimp*, the environment processes solution *eps*
Ensure: the optimized solution set HIS of *Aimp*
1: **while** the termination condition of IMDE is not true **do**
2: **if** this is the first iteration of IMDE **then**
3: Set $CEP = IP$ and $HIS = \emptyset$
4: Initially selects $x_{CEP}^{Rand}, x_{HIS}^{Rand}$ randomly from CEP
5: **else**
6: Randomly selects $x_{CEP}^{Rand}, x_{HIS}^{Rand}$ from CEP and HIS, respectively
7: **end if**
8: $x_{CEP}^{Best} = maxfit(CEP, eps)$
9: $x_{HIS}^{Best} = maxfit(HIS, eps)$
10: Create $AGS = \{x_{CEP}^{Rand}, x_{CEP}^{Best}, x_{HIS}^{Rand}, x_{HIS}^{Best}\}$
11: $BCMAGS = BM(CR(MU(AGS)))$
12: Create $CGS = AGS \cup BCMAGS$
13: $x_{CGS}^{Best} = maxfit(CGS, eps)$
14: **if** fitness of x_{CGS}^{Best} > fitness of x_{CEP}^{Worst} **then**
15: update CEP by replacing x_{CEP}^{Worst} by x_{CGS}^{Best}
16: **end if**
17: Update HIS as update CEP by comparing x_{CGS}^{Best} and x_{HIS}^{Worst}
18: **end while**

Part 1: The first part creates the aim gene set (*AGS*). The traditional DE randomly selects genes or gets the best gene in the current population to mutate and cross over, so that the optimal solution comes from the same parent population. Algorithm 2 (lines 2-10) shows how different types of populations are used to serve as the aim genes, to bring richness to the evolution.

In the first iteration, the current population (*CEP*) is the initial population (*IP*), and the historically optimized population (*HIS*) is an empty set. The random strategy genes, x_{CEP}^{Rand} and x_{HIS}^{Rand}, were randomly selected from the *CEP* and the *HIS* for the diversity of population. The best strategy genes, x_{CEP}^{Best} and x_{HIS}^{Best}, are the best gene of the *CEP* and the *HIS* for evolutionary search efficiency. The *AGS* is composed of the above four types of genes.

Part 2: The second part creates the comparison gene set (*CGS*). Algorithm 2 (lines 11-12) shows that the evolutionary genes *BCMAGS* are obtained by the mutating (*MU*), crossing (*CR*), and boundary-modifying (*BM*) of the *AGS*. We combine *BCMAGS* and original *AGS* to create the *CGS* for finding the best gene of the *CEP*.

Traditional mutation strategies are adopted for *AGS* to get the corresponding mutated gene u, including two random strategies (DE/rand/1 and DE/rand/2) and two best strategies (DE/best/1 and DE/best/2). These strategies make full use of the solution genes $(x^{r1}, x^{r2}, x^{r3}, x^{r4})$ in the current population (*CEP*) to obtain the mutation segment under a scaling control parameter F, $F(x^{r1} - x^{r2})$ or $F(x^{r1} - x^{r2}) + F(x^{r3} - x^{r4})$. Thus, the random gene x^{Rand} or the optimal gene x^{Best} in *CEP* is mutated by using these mutation segments. In the improvement of DE, we wanted to increase the diversity of candidate mutation genes, to increase the possibility of obtaining excellent mutation genes. In addition to performing these mutation strategies in *CEP*, these mutation strategies were also performed in *HIS*. Therefore, the mutation strategies (*MU*) are expressed as Eq. 15:

$$u = \begin{cases} x_{CEPorHIS}^{Rand} + F(x^{r1} - x^{r2}) & , DE/rand/1 \\ x_{CEPorHIS}^{Rand} + F(x^{r1} - x^{r2}) + F(x^{r3} - x^{r4}) & , DE/rand/2 \\ x_{CEPorHIS}^{Best} + F(x^{r1} - x^{r2}) & , DE/best/1 \\ x_{CEPorHIS}^{Best} + F(x^{r1} - x^{r2}) + F(x^{r3} - x^{r4}) & , DE/best/2 \end{cases}. \tag{15}$$

A traditional binominal crossover operator (*CR*) is used to cross the aim gene x and its mutated gene u. For the j^{th} subtask, crossover is represented as follows:

$$v_j = \begin{cases} u_j & , \text{if } rand(0,1) \le CR \text{ or } j = j_{rand} \\ x_j^{Rand} \text{ or } x_j^{Best} & , \text{otherwise} \end{cases}. \tag{16}$$

where u_j is the j^{th} gene position of u and j_{rand} is a random position of the gene; $CR \in [0, 1]$ is the crossover rate, which describes the copy fraction from the mutated gene; v is the crossed gene; and v_j is the j^{th} gene position of v.

After mutation and crossover, the value of some gene positions may exceed the range of the candidate cloud service sets of the subtask. The boundary-modifying (*BM*) is designed to ensure that genes can get a reasonable serial number of cloud services. There are sn_j cloud services for the j^{th} subtask, and thus the serial number scale is $[1, sn_j]$:

$$bmv_j = \begin{cases} sn_j - mod(1 - v_j, sn_j - 1) & , \text{if } v_j \le 1 \\ v_j & , \text{if } 1 < v_j < sn_j \\ 1 + mod(v_j - sn_j, sn_j - 1) & , \text{if } v_j \ge sn_j \end{cases}. \tag{17}$$

where *mod* is the mathematical modulo operation, BMV is the genetic border modified gene, and bmv_j is the j^{th} position in this gene.

Mutation, crossover, and boundary-modifying make *AGS* evolve into *BCMAGS*. The *CGS* is constructed as $AGS \cup BCMAGS$, in which genes are used to find the best gene in this iteration.

Part 3: The third part involves updating the *CEP* and *HIS*. Algorithm 2 (lines 13-17) shows that the best gene is found in the *CGS* of this iteration, and the *CEP* and *HIS* are updated by this best gene.

The whole multi-process fitness value is calculated by combining the *CGS* with *eps*. The highest fitness value gene x_{CGS}^{Best} is selected as the best gene in this iteration. x_{CGS}^{Best} replaces the worst fitness value gene in the *CEP*, x_{CEP}^{Worst}. If the fitness value of the former is higher than that of the latter, the *CEP* is updated. Similarly, the *HIS* is updated by x_{CGS}^{Best}. Finally, the optimal solution set and the best gene are obtained in the *HIS*.

4.1.3 Strategies for Handling Constraints

For every subtask in the process, the selected cloud service must satisfy the lowest threshold $\widetilde{nq}_{c,ST_{ij}}$ for each of the five QoS characteristics; this threshold is determined based on the requirements of the subtask. Each subtask ST_{ij} in the process P_i has a variety of candidate cloud services with their own QoS characteristics provided by the service providers or private services developed by the organization itself, which also has corresponding QoS characteristics. Due to the minimum QoS requirements of the subtask, the selected cloud service needs to meet the QoS threshold of the subtask. Therefore, it is necessary to filter the candidate cloud services of the subtask to retain only the services that meet the corresponding QoS threshold of the subtask. This process ensures that all candidate cloud services meet the subtask QoS constraint. Finally, the candidate cloud services that are retained enter the optimization process to select a cloud service that best meets the optimization objective.

For each process, the selected cloud service should also meet the process QoS constraint $\widetilde{nq}_{c,P_i}$. We use an adaptive penalty method to solve process QoS constraints (de Melo and Iacca, 2014). The fitness value $Fit(x)$ of each gene x is calculated using Eq. 18 to obtain the penalty adjusted fitness value $F\_P(x)$.

$$F\_P(x) = \begin{cases} Fit(x) & \text{, if } x \text{ is } FS \\ BESTF - \mid BESTF - Fit(x) \mid\mid \frac{Vio(x)}{maxVio} \mid & \text{, otherwise} \end{cases}. \tag{18}$$

If there are feasible solutions (FS), $BESTF$ is the best feasible solution fitness value; otherwise $BESTF$ is the best infeasible solution fitness value. $Vio(x)$ is the extent of violation of the process QoS constraint, which is calculated by Eq. 19. $maxVio$ is the largest violation of all infeasible solutions.

$$Vio(x) = \begin{cases} 0 & \text{, if } nq_{c,P_i} \geq \widetilde{nq}_{c,P_i} \\ \sum_c \omega_c \frac{\widetilde{nq}_{c,P_i} - nq_{c,P_i}}{\widetilde{nq}_{c,P_i}} & \text{, otherwise} \end{cases}. \tag{19}$$

This method imposes a greater penalty on the infeasible solution, which more seriously violates the process QoS constraint. This method guarantees that an infeasible solution with a small violation has a high fitness value and is more likely to be retained in the evolutionary set to keep population diversity. However, since the fitness value of this infeasible solution cannot be higher than that of the best feasible solution in the population, it will not influence the genetic evolution of the best feasible solutions.

For the implicit common constraint (Eq. 14), boundary corrections need to be made at each iteration of the algorithm using Eq. 17 to ensure that the subtask decision variable satisfies the constraint.

4.2 Analysis of Computational Complexity

4.2.1 Data Instance and Model Complexity

The computational complexity of multi-factor CSCOS is related to the number of processes, the number of subtasks in the process, and the number of candidate cloud services for the subtasks. In general, a large enterprise usually has dozens or even hundreds of business processes, while a small organization may have only a few business processes. However, the model needs to focus on only related business processes. The subtasks in the business process depend on the complexity of the business and the segments involved. The number of cloud services available for the subtask depends on the nature of the subtask. A general subtask might have many candidate services, while a specialized subtask might have few candidate services. Thus, the scale of the problem is case dependent.

If a problem has m processes, assuming that the number of subtasks in each process is n, and the average number of candidate cloud services for each subtask are l, this problem is denoted by the notation "PmTnSl", and the size of the problem is $m \cdot n \cdot l$. Although the solution space is a finite discrete set, there are l^{nm} feasible solutions in this set; thus, finding the optimal solution in the set is an NP-hard problem. Just as in previous studies for approximate solutions, this paper develops a heuristic evolutionary algorithm, C3IMDE.

4.2.2 Computational Complexity of C3IMDE

C3IMDE is divided into the inner IMDE algorithm and the outer C3 framework. The computational complexity of dif-

ferential evolution is captured by the product of the number of objective function evaluations, the population size, and the maximum number of iterations (Opara et al, 2019). Let the population size and the maximum number of iterations of the IMDE proposed in this paper be $NIMDE_p$ and $GIMDE^{max}$, respectively. The total number of subtasks of all processes is the dimension of the solution space as $m \cdot n$ for the problem "PmTnSl", which is the number of the subtask decision variables and the sum of the lengths of all process genes. And the objective function evaluation $c(m \cdot n)$ is at most linearly dependent on the search space dimension $m \cdot n$. Therefore, the computational complexity of IMDE, $CCoIMDE = O(c(m \cdot n) \cdot NIMDE_p \cdot GIMDE^{max})$. Let the maximum number of iterations in stage 1 and stage 2 of C3 be $GC3S1^{max}$ and $GC3S2^{max}$. According to the pseudo code of C3, the computational complexity of C3IMDE is $O(GC3S1^{max} \cdot (GC3S2^{max} \cdot CCoIMDE + CCoIMDE))$.

Table 6 Parameters of algorithms

Algorithm	Description of parameters
C3IMDE	In the *CEPSS* and *CMPSS*, the number of stored solutions for each process is five, and the parameters of the inner layer optimization algorithm are set to the IMDE algorithm parameters.
DE, IMDE	The mutation factor F and the crossover probabilities CR were randomly generated using the scales [0.2 0.5] and [0.1 0.5].
GA, IMGA	IMGA improves GA by adding process crossover and node crossover. The crossover probabilities of the process, fragment, and node are 0.8, 0.5, and 0.2, respectively, and the mutation probability is 0.1.
PSO	The inertia weight is 0.8, and the self-learning factor and group-learning factor are each 0.5.

5 Results of Experiments and Discussion

5.1 Synthetic Data Generation

Due to the complexity of the research problem, it is difficult to obtain real data that conform to the service composition optimization model. Because of this, most researchers in this area use simulation to generate synthetic data (Asghari and Navimipour, 2019; Jin et al, 2017; Yang et al, 2019; Zhou and Yao, 2017a); this paper also follows this practice, and combines generality and randomness to generate simulated data that represent random real scenarios without a loss of generality.

Regarding the type, brand, and special requirements, based on the number of candidate cloud services for each subtask, the proportions of each type of service are set. For example, the proportion of OPC, SCS, and GPC types of services is (0.2, 0.2, 0.6). For each candidate cloud service, the proportion of brands is set; for example, the total number of brands of OPC, SCS, and GPC services on a subtask with 10 candidate cloud services is (1, 2, 7). The brand is assigned to the corresponding type of cloud service product by randomly selecting a number from 1 to the total number of brands. Subtasks requiring special requirements are randomly assigned according to the number of subtasks in the process.

Regarding the value of QoS characteristics, the value of the five QoS characteristics (availability, reliability, cost, time, reputation) is based on three cloud service types (OPC, GPC, SCS). We assume that a private cloud service (OPC) has a high-quality availability, low-quality reliability, low-quality cost, high-quality time, and low-quality reputation. A special cloud service (SCS) has a high-quality availability, high-quality reliability, high-quality cost, low-quality time, and high-quality reputation. The QoS characteristics of general public cloud (GPC) are between those of OPC and SCS and are of medium quality. We randomly generate data for each QoS characteristic using a normal distribution. Mean and variance follow the principles: high quality has a high mean and low variance, low quality has a low mean and high variance, and medium quality is between high quality and low quality. These values conform to the range of QoS characteristics in Table 3.

5.2 Experimental Analysis and Discussion

To compare the performance of our model and solution algorithm, a set of test cases was developed, and the results are compared with those of other optimization algorithms. For fairness, we set the weight of *TQoS*, *TSR*, and *TSC* as 1, and the weight of the five QoS characteristics as $\omega_c = 1/5$. The degree of improvement of subtask QoS standard for a process QoS standard was set as $\alpha_{c,P_i} = 5\%$. The $Node$ and $Side$ cumulative effect was $\lambda_{Node} = \lambda_{Side} = 1$. The parameters used for each algorithm are shown in Table 6. The experimental environment was a Windows 10 OS on a laptop with an Intel Core I5-4200M CPU and 12G RAM. The proposed algorithm was programmed in MATLAB R2016a.

5.2.1 Comparison with a Single Process

To compare the performance of various algorithms on a single process, we conducted two sets of experiments. In the first set of experiments, the number of subtasks was fixed at $n = 20$, and the average number of candidate cloud services for each subtask was varied between 10 and 500, that is, l is 10, 15, 20, 30, 100, and 500, respectively. Thus, the scale of the multi-factor CSCOS varied from 200 to 10,000

candidate cloud services. The average optimal fitness values for different algorithms are shown in Fig. 4 (1). For six test cases, the C3IMDE algorithm had the highest average optimal fitness value, and the performance of IMDE was second after C3IMDE. The average optimal fitness value of C3IMDE was 2.5147, and for the IMDE, DE, IMGA, GA, and PSO algorithms, the values were 2.5025, 2.2738, 2.4497, 2.3642, and 1.5337, respectively. Although the efficiency of all the algorithms decreased as the scale of multi-factor CSCOS increased, the C3IMDE and IMDE algorithms had a slower decline and the best stability.

In the second set of experiments, the average number of candidate cloud services for each subtask was fixed at $l = 30$. The number of subtasks in the six test cases varied from 10 to 500, that is, n is 10, 15, 20, 30, 100, and 500, respectively. Thus, the multi-factor CSCOS scale varied from 300 to 15,000 candidate cloud services. The average optimal fitness values for various algorithms are shown in Fig. 4 (2). Again, the C3IMDE and IMDE algorithms had the highest and second-highest average fitness value. The average optimal fitness value of C3IMDE was 2.2387, and IMDE, DE, IMGA, GA, and PSO were 2.2050, 1.9514, 2.0959, 1.9765, and 1.4707, respectively. These results are similar to the experiment with a fixed number of subtasks. Note that the change in the number of subtasks had a greater impact on the algorithms than the change in the number of cloud services available for a subtask.

In general, the proposed algorithm could find the solution with a higher fitness value than other algorithms, and the fitness value of the solution fluctuated less for the proposed algorithm than with other algorithms, as the scale of the problem changed. Thus, the C3IMDE algorithm had the best performance and better stability in the single-process multi-factor CSCOS. Note that in one process situation, C3IMDE used only the second stage of C3, so C3IMDE was essentially working as IMDE. However, since the initial population of each cycle was set as the last generation optimal solution set, C3IMDE was better than IMDE in fitness results as the scale grew. In addition, IMDE had the second-best performance. Thus, it is reasonable to select the IMDE as the inner layer optimization algorithm of C3.

5.2.2 Comparison with Multiple Processes

To test the performance of the proposed algorithm in multi-process environments, we generated nine groups of test cases.

Figure 5 shows that as the problem size increased, the optimal fitness values obtained by all algorithms got smaller. However, C3IMDE had the best performance in all test cases. The average optimal fitness value dropped only from 2.5778 in the case "P3T10S10" to 2.3520 in the case "P10T30S30." For other algorithms, the fitness values were significantly lower. Although the performance of IMDE was second only to that of C3IMDE, there was a big gap between them. The average optimal fitness value of IMDE varied from 2.5206 for "P3T10S10" to 1.6872 for "P10T30S30." The DE, IMGA, GA, and PSO algorithms' performances were lower. Also,

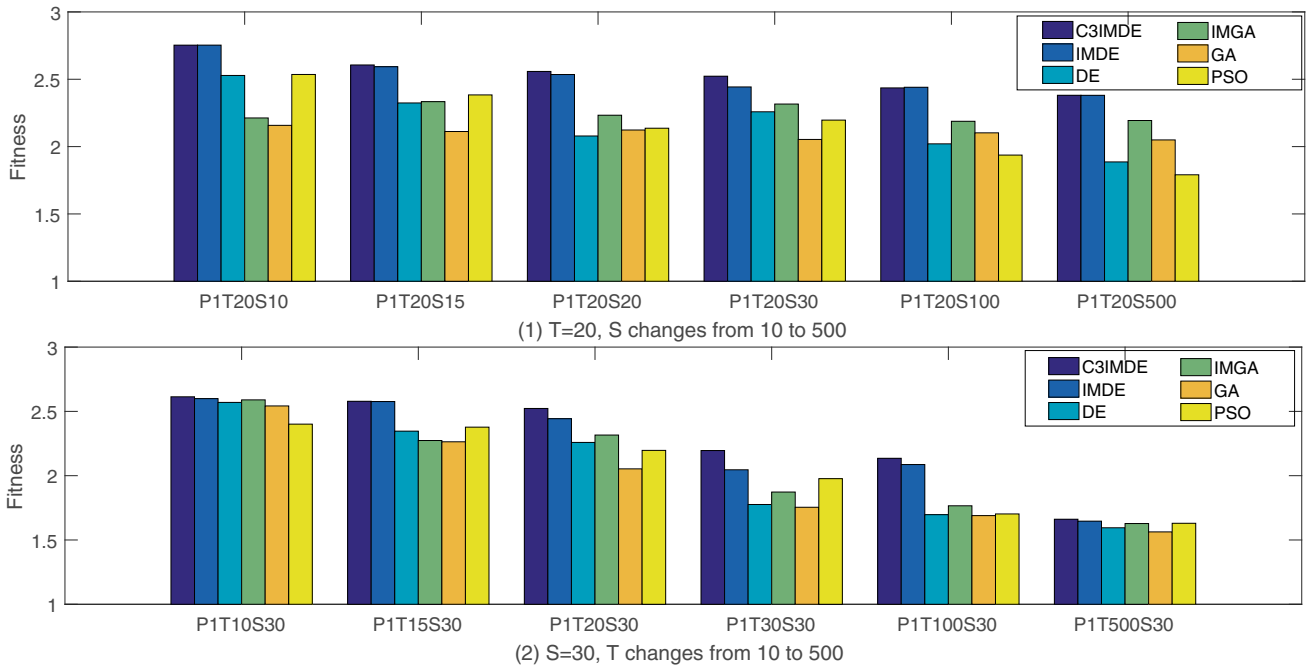

Fig. 4 Optimization algorithm comparison with a single process

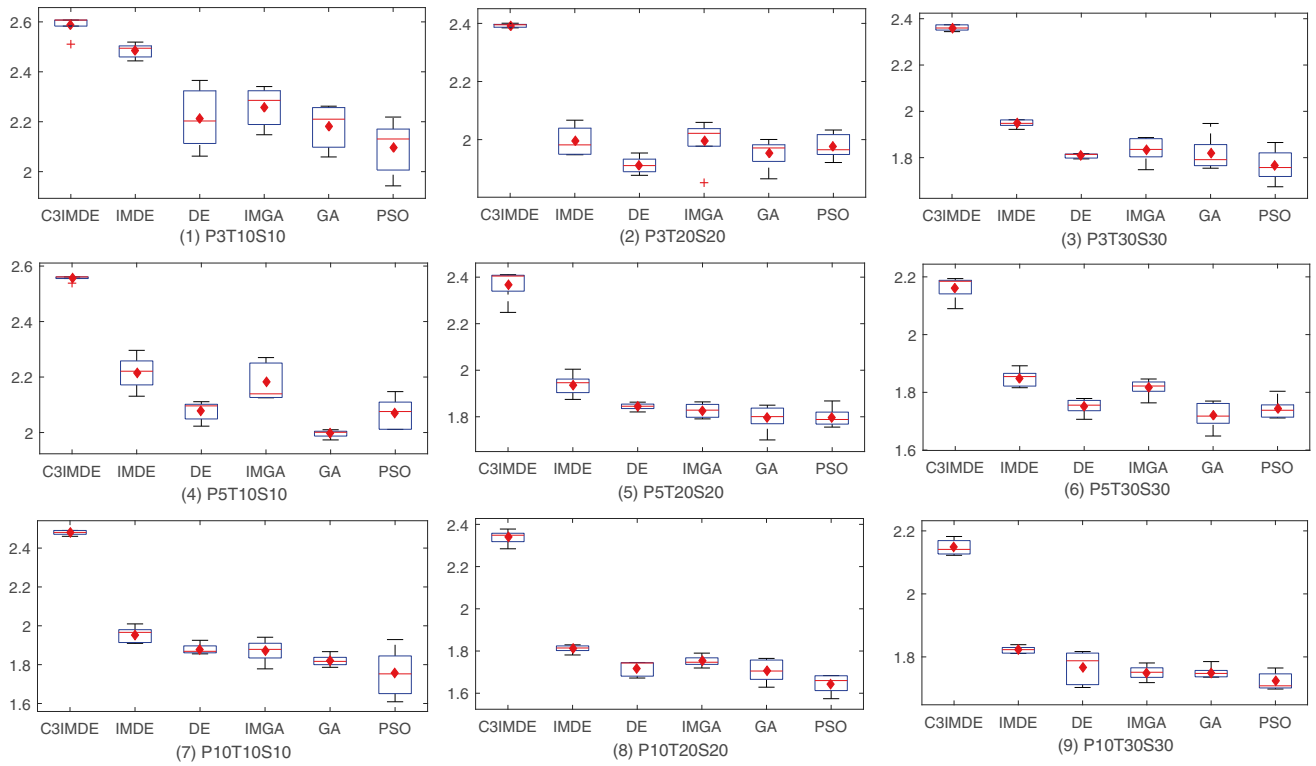

Fig. 5 Optimization algorithm comparison with multiple processes

the fluctuation of the optimal fitness value of C3IMDE was the smallest, which is shown by the minimum height of the box diagram.

In general, C3IMDE had better efficiency and stability in the multi-process multi-factor CSCOS. The performance was significantly better than that of other algorithms, especially for larger problem sizes. The C3IMDE algorithm had a higher fitness value of the solution and a smaller fluctuation than other traditional algorithms.

5.2.3 Comparison of Different Weights of Objective Function

In order to analyze the impact of different weight setting preferences on the results in the objective function, this paper chooses a single-process example "P1T30S30" and a multi-process example "P3T30S30" to conduct numerical experiments. We set $\omega_1 : \omega_2 : \omega_3$ to several different values. The solution results are shown in Tables 7 and 8.

Table 7 Results of $TQoS$, TSR, and TSC under different weight ratios of "P1T30S30"

M	$\omega_1 : \omega_2 : \omega_3 = M : 1 : 1$				$\omega_1 : \omega_2 : \omega_3 = 1 : M : 1$				$\omega_1 : \omega_2 : \omega_3 = 1 : 1 : M$			
	$TQoS$	TSR	TSC	$TOTAL$	$TQoS$	TSR	TSC	$TOTAL$	$TQoS$	TSR	TSC	$TOTAL$
1	0.6729	0.8750	0.7562	2.3041	0.6729	0.8750	0.7562	2.3041	0.6729	0.8750	0.7562	2.3041
	29.20%	37.98%	32.82%		29.20%	37.98%	32.82%		29.20%	37.98%	32.82%	
2	0.6867	0.8750	0.7788	2.3405	0.6829	0.8750	0.7827	2.3406	0.8750	0.8750	0.8112	2.5612
	29.34%	37.39%	33.28%		29.18%	37.38%	33.44%		27.57%	37.58%	34.84%	
3	0.6878	0.8750	0.7526	2.3154	0.6838	0.8750	0.7827	2.3415	0.6504	0.7500	0.8337	2.2341
	29.70%	37.79%	32.50%		29.20%	37.37%	33.43%		29.11%	33.57%	37.32%	
4	0.6942	0.8750	0.7533	2.3225	0.6829	0.8750	0.7827	2.3406	0.6483	0.6250	0.8755	2.1488
	29.89%	37.67%	32.43%		29.18%	37.38%	33.44%		30.17%	29.09%	40.74%	
5	0.7206	0.8750	0.6121	2.2077	0.6588	0.8750	0.7347	2.2685	0.6490	0.6250	0.8755	2.1495
	32.64%	39.63%	27.73%		29.04%	38.57%	32.39%		30.19%	29.08%	40.73%	

Table 8 Results of $TQoS$, TSR, and TSC under different weight ratios of "P3T30S30"

M	$\omega_1 : \omega_2 : \omega_3 = M : 1 : 1$				$\omega_1 : \omega_2 : \omega_3 = 1 : M : 1$				$\omega_1 : \omega_2 : \omega_3 = 1 : 1 : M$			
	$TQoS$	TSR	TSC	$TOTAL$	$TQoS$	TSR	TSC	$TOTAL$	$TQoS$	TSR	TSC	$TOTAL$
1	0.6944	1.0000	0.7922	2.4866	0.6944	1.0000	0.7922	2.4866	0.6944	1.0000	0.7922	2.4866
	27.93%	40.22%	31.85%		27.93%	40.22%	31.85%		27.93%	40.22%	31.85%	
2	0.7278	1.0000	0.7293	2.4571	0.6846	1.0000	0.7906	2.4752	0.6681	1.0000	0.8158	2.4647
	29.62%	40.70%	29.68%		27.66%	40.40%	31.94%		27.11%	40.57%	32.32%	
3	0.7410	1.0000	0.7335	2.4745	0.7094	1.0000	0.7914	2.5008	0.6226	1.0000	0.8163	2.4308
	29.95%	40.41%	29.64%		28.37%	39.99%	31.65%		25.61%	41.14%	33.25%	
4	0.7440	1.0000	0.7131	2.4571	0.6892	1.0000	0.8048	2.4940	0.6595	1.0000	0.8203	2.4698
	30.28%	40.70%	29.02%		27.63%	40.10%	32.27%		26.70%	40.49%	32.81%	
5	0.7569	1.0000	0.6731	2.4300	0.6884	1.0000	0.8146	2.5030	0.6085	1.0000	0.8166	2.4251
	31.15%	41.15%	27.70%		27.50%	39.95%	32.54%		25.09%	41.24%	33.67%	

As shown in Table 7, as the weights of $TQoS$ and TSC get larger, the solution scores of $TQoS$ and TSC increase accordingly and their proportion in the total score ($TOTAL = TQoS + TSR + TSC$) also increases. However, the TSR values in the solution stay almost the same in most cases. This is due to the fact that there are fewer SRSTs in the process, and a SCS-type service has better QoS characteristic values in the simulated data set, so all SRSTs can be more easily optimized to obtain a SCS-type service, unless there are no SCS-type service in the SRST's candidate cloud services (as in the case of the simulated data for the experiment result in Table 7, there are 8 SRSTs in the process, but one of SRSTs has no SCS-type service among its candidate cloud services, so that $TSR = \frac{7}{8} = 0.875$ is already the highest score). In addition, the results of TSR are also affected when the weight of TSC gets larger. This shows that as the weight preference of TSC is set larger and larger, TSR will give way to the optimization of TSC, even if SRST is easily satisfied. The results of working with multiple processes (Table 8) are similar.

The experiment shows that the results of the optimal solution are sensitive to the choice of weights. This illustrates that the model and algorithm proposed in this paper can enable decision makers of CSCOS to control the optimization results to meet the objectives and preferences of the organization by setting different weights.

Tables 7 and 8 show that a relatively balanced weight ratio results in a higher total score, whereas a biased weight ratio lowers the total score. That is, the weighting preferences are not favorable to the total score. This is due to the fact that when searching for the optimal solution based on the three aspects ($TQoS$, TSR, TSC), an increase of the weight preference setting of one aspect affects the balance of the other two aspects. Therefore, although a search result that matches the decision preferences can be obtained through a high weight setting, this result is not optimal from an overall perspective, and as the preference weights increase, the greater the impact on the overall optimal result. Thus, the weight setting should consider weight ratio balance, unless the decision maker's preference for one aspect is more important than the overall score.

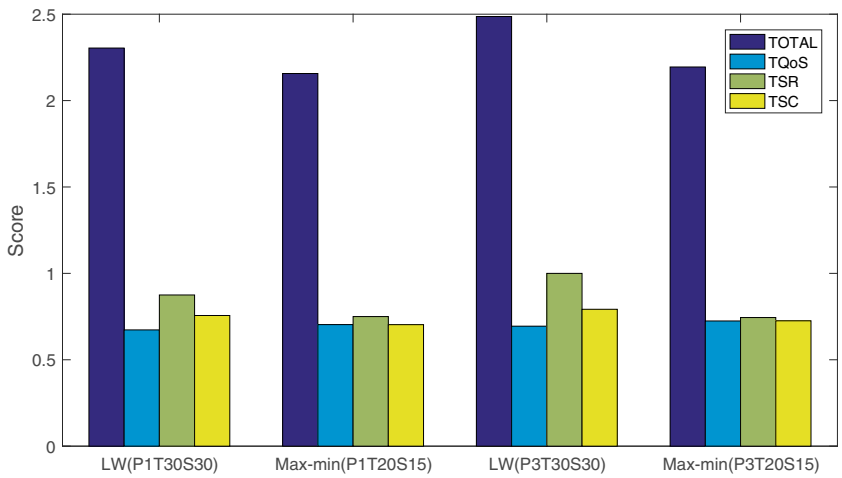

Fig. 6 Comparison of objective functions LW and Max-min

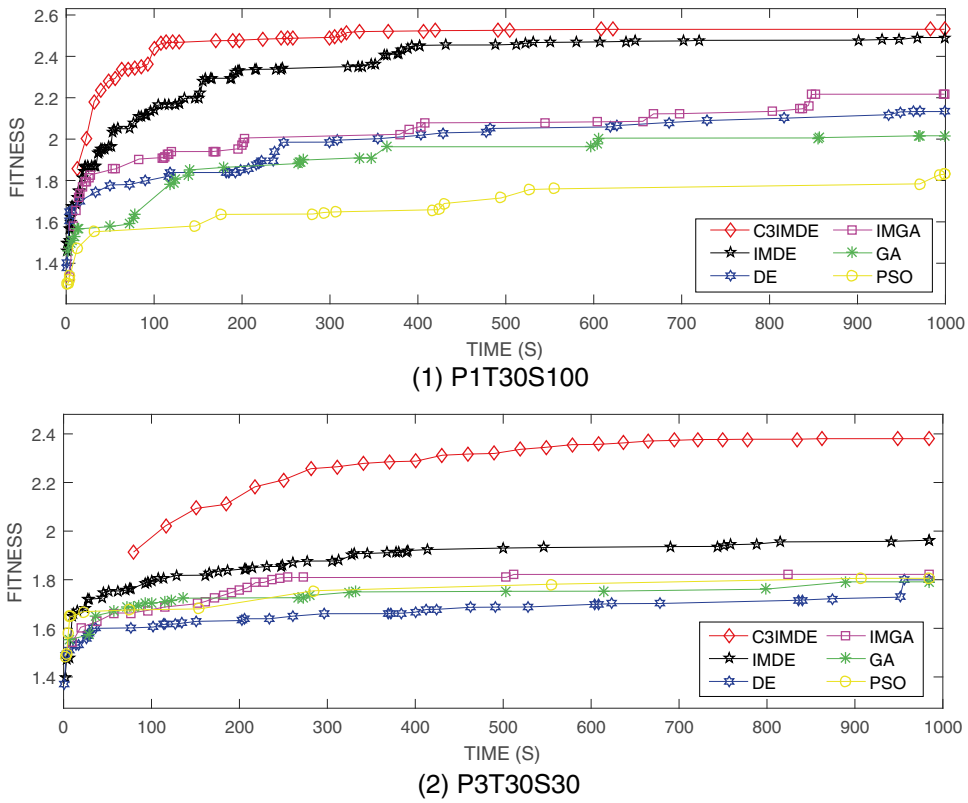

Fig. 7 Optimal fitness value changes with time

5.2.4 Comparison of Two Objective Functions

As discussed above, the linear weighted (LW) objective function used in this paper is easily affected by the preference settings of decision makers. Usually, the objective function that maximizes the minimum value (Max-min) of $TQoS$, TSR, and TSC can achieve a robust objective. We chose a single-process example, "P1T30S30," and a multi-process example, "P3T30S30," to conduct numerical experiments to compare the two objective functions.

We set the preference weight ratio of the LW objective function to 1:1:1. The experimental results (Fig. 6) show that in the single-process "P1T30S30," the scores of $TOTAL$, $TQoS$, TSR, and TSC for LW are 2.3041, 0.6729, 0.8750, and 0.7562, while for the Max-min setting the scores are 2.1567, 0.7035, 0.7500, and 0.7032, respectively. In terms of $TOTAL$ score, TSR score, and TSC score, LW is significantly better than Max-min, while in $TQoS$ score, LW is slightly worse than Max-min. In terms of the percentage of $TQoS$, TSR, and TSC, LW is 29.20%, 37.98%, and 32.82% and Max-min is 32.26%, 34.78%, and 32.61%. Figure 6 shows the comparison. The above situation is more obvious in the multi-process example, "P3T30S30."

Therefore, Max-min is better than LW in terms of robust balancing, while LW is better than Max-min in the search for optimal solutions. In addition, another benefit of LW is the flexibility to formulate CSCOS decision preferences.

5.2.5 Comparison of Time Efficiency

To analyze the time efficiency of the algorithms, we executed experiments on single-process "P1T30S100" and multi-process "P3T30S30."

Table 9 Execution time of IMDE and C3IMDE for a single process

	$IMDE$	$C3IMDE$		$IMDE$	$C3IMDE$
P1T20S10	0.2019	3.4489	P1T10S30	0.1186	1.6113
P1T20S15	0.2124	2.9601	P1T15S30	0.1525	2.4838
P1T20S20	0.2237	4.5151	P1T20S30	0.1770	3.2917
P1T20S30	0.1770	3.2917	P1T30S30	0.3236	6.4271
P1T20S100	0.1883	3.3913	P1T100S30	2.0954	49.1511
P1T20S500	0.2545	3.8811	P1T500S30	18.4226	419.1132

Table 10 Execution time of IMDE and C3IMDE for multiple processes

P	$IMDE$			$C3IMDE$		
	T10S10	T20S20	T30S30	T10S10	T20S20	T30S30
3	0.3668	0.5304	0.9764	12.3773	26.5403	49.1961
5	0.4150	0.9050	1.5662	21.5170	50.5176	84.6455
10	0.8937	1.5945	2.6478	51.3188	102.3062	166.0371

Figure 7 shows that although C3IMDE took longer to obtain the first optimal fitness value (because CEPSS needed to be created in stage 1 of the C3 framework), the first optimal fitness value was better than those of other algorithms. The fitness value of 1.3904 was obtained in 6.708 seconds for the test case "P1T30S100," and 1.77 was obtained in 48.235 seconds for the test case "P3T30S30." The performance curve for the C3IMDE algorithm was always above the performance curves of all the other algorithms. Thus, C3IMDE can find the solution with a higher fitness value at the same time as the other algorithms, indicating better time efficiency.

5.2.6 Comparison of Computational Effort

To analyze the computational effort of C3IMDE, experiments were conducted for single-process and multi-process examples for different problem sizes to obtain the mean values of the execution time of IMDE and C3IMDE at the first iteration as shown in Tables 9 and 10.

Tables 9 and 10 show that the increase in both the number of processes and the number of subtasks causes the iteration time of IMDE and C3IMDE to grow, and the C3 framework amplifies this effect by calling IMDE multiple times. However, the increase in the number of candidate cloud services for the subtasks has a smaller effect on the iteration time. This fact also validates the results of the computational complexity in Section 4.2.2, because the CCoIMDE correlates with the search space dimension (the product of the number of processes and the number of subtasks) and does not correlate with the number of candidate cloud services or the number of subtasks.

6 Conclusions and Future Research Directions

Multi-factor CSCOS is an important problem when selecting cloud services for subtasks in business processes. This is especially true in the hybrid cloud environment, which is composed of various types and brands of cloud services. Multiple collaborative processes are needed to maintain cloud service consistency and high QoS performance while meeting the special requirements of some tasks. These needs make multi-factor CSCOS very complex. This paper constructs a constrained combinatorial optimization model and develops the C3IMDE algorithm to solve it. Experiments show that C3IMDE has a higher optimal solution search efficiency and stability than the DE, GA, and PSO models for different sizes of multi-factor CSCOS.

This paper considers only a single-objective optimization. However, there is an inherent relationship between QoS performance, special requirements, and cloud service consistency, so multi-objective methods could be used for further analysis. In addition, it is worth studying complex business workflows (e.g., there are multiple items being processed at the same time in the same process through different process subtasks in various business logic situations), finer-grained business queues (e.g., split queues for running a business on subtasks, and the priority and waiting details of each queue), and a cloud service's server execution details (e.g., multiple server collaboration of the cloud service running on a subtask, as well as the resource allocation and scalability issues at runtime).

This paper considered the QoS values from a static perspective, but in practice the QoS values change dynamically with the operation of the cloud service; This dynamic perspective can be considered using dynamic programming to model it. Moreover, this paper studies only the sequential structure of the subtask in the business process, which has certain limitations (the impact of scalability, metrics for maximum duration under parallel conditions, etc.). Since there can be more complex structures of business processes that are difficult to translate into sequential patterns in real-world problems, especially in the field of elastic business processes, the measurement of QoS and consistency of the cloud service under these structures needs to be studied in future research.

Finally, cloud-based BPM research can use cloud computing technology to achieve elastic and agile business processes in real scenarios, and it is necessary to go deep into the details of cloud computing technology in the execution of these business processes. Therefore, it will be very valuable to consider the QoS performance measurement method of dynamic resource scheduling and allocation, and the technical details of the consistency between special cloud services (such as NoOps serverless computing) and common cloud services, and how they affect BPM.

Acknowledgements This work was supported by the National Natural Science Foundation of China under grants 72131006, 71771075, 72271082, and 72071063. Natural Science Foundation of Universities of Anhui Province under grant KJ2021A0473, Anhui Provincial Key Research and Development Plan Project under grant 2022i01020003 and Fundamental Research Funds for the Central Universities under grant PA2020GDKC0020.

Data Availability The data that support the findings of this study is synthetic data generated by the approach described in this paper, and the data for all analyses is available upon request.

Declarations

Conflicts of interest The authors have no conflicts of interest to declare that are relevant to the content of this article.

References

Adhikari, M., Amgoth, T., & Srirama, S. N. (2019). A survey on scheduling strategies for workflows in cloud environment and emerging trends. *ACM Computing Surveys, 52*(4), 1–36. https://doi.org/10.1145/3325097

Aghamohammadzadeh, E., & Valilai, O. F. (2020). A novel cloud manufacturing service composition platform enabled by blockchain technology. *International Journal of Production Research, 58*(17), 5280–5298. https://doi.org/10.1080/00207543.2020.1715507

Asghari, S., & Navimipour, N. J. (2019). Cloud service composition using an inverted ant colony optimisation algorithm. *International Journal of Bio-Inspired Computation, 13*(4), 257–268. https://doi.org/10.1504/ijbic.2019.100139

Bülbül, K., Noyan, N., & Erol, H. (2021). Multi-stage stochastic programming models for provisioning cloud computing resources. *European Journal of Operational Research, 288*(3), 886–901. https://doi.org/10.1016/j.ejor.2020.06.027

Chen, L., & Chang, W. (2020). Under what conditions can an application service firm with in-house computing benefit from cloudbursting? *European Journal of Operational Research, 282*(1), 71–80. https://doi.org/10.1016/j.ejor.2018.11.016

Chen, Y., Huang, J., Lin, C., & Shen, X. (2019). Multi-objective service composition with QoS dependencies. *IEEE Transactions on Cloud Computing, 7*(2), 537–552. https://doi.org/10.1109/tcc.2016.2607750

Cohen, M. C., Keller, P. W., Mirrokni, V., & Zadimoghaddam, M. (2019). Overcommitment in cloud services: Bin packing with chance constraints. *Management Science, 65*(7), 3255–3271. https://doi.org/10.1287/mnsc.2018.3091

Deng, S., Wu, H., Hu, D., & Zhao, J. L. (2016). Service selection for composition with QoS correlations. *IEEE Transactions on Services Computing, 9*(2), 291–303. https://doi.org/10.1109/tsc.2014.2361138

Ding, S., Wang, Z., Wu, D., & Olson, D. L. (2017). Utilizing customer satisfaction in ranking prediction for personalized cloud service selection. *Decision Support Systems, 93*, 1–10. https://doi.org/10.1016/j.dss.2016.09.001

Drake, N. (2014). Cloud computing beckons scientists. *Nature, 509*(7502), 543–544. https://doi.org/10.1038/509543a

Euting, S., Janiesch, C., Fischer, R., Tai, S., & Weber, I. (2014). *Scalable business process execution in the cloud* pp 175-184. Boston, MA, USA: 2014 IEEE International Conference on Cloud Engineering (IC2E). https://doi.org/10.1109/IC2E.2014.13

Fox, A. (2011). Cloud computing-what's in it for me as a scientist? *Science, 331*(6016), 406–407. https://doi.org/10.1126/science.1198981

Gabrel, V., Manouvrier, M., Moreau, K., & Murat, C. (2018). QoS-aware automatic syntactic service composition problem: Complexity and resolution. *Future Generation Computer Systems-the International Journal of eScience, 80*, 311–321. https://doi.org/10.1016/j.future.2017.04.009

Glorieux, E., Svensson, B., Danielsson, F., & Lennartson, B. (2017). Constructive cooperative coevolution for large-scale global optimisation. *Journal of Heuristics, 23*(6), 449–469. https://doi.org/10.1007/s10732-017-9351-z

Gu, D. X., Deng, S. Y., Zheng, Q., Liang, C. Y., & Wu, J. (2019). Impacts of case-based health knowledge system in hospital management: The mediating role of group effectiveness. *Information & Management, 56*(8), 1–12. https://doi.org/10.1016/j.im.2019.04.005

Hoenisch, P., Hochreiner, C., Schuller, D., Schulte, S., Mendling, J., & Dustdar, S. (2015). *Cost-efficient scheduling of elastic processes in hybrid clouds* (pp 17-24). New York City, NY, USA: IEEE 8th International Conference on Cloud Computing (CLOUD). https://doi.org/10.1109/CLOUD.2015.13

Hoenisch, P., Schuller, D., Schulte, S., Hochreiner, C., & Dustdar, S. (2016). Optimization of complex elastic processes. *IEEE Transactions on Services Computing, 9*(5), 700–713. https://doi.org/10.1109/TSC.2015.2428246

Jain, T., & Hazra, J. (2019). Hybrid cloud computing investment strategies. *Production and Operations Management, 28*(5), 1272–1284. https://doi.org/10.1111/poms.12991

Janiesch, C., Weber, I., Kuhlenkamp, J., & Menzel, M. (2014). *Optimizing the performance of automated business processes executed on virtualized infrastructure* pp. 3818-3826. Waikoloa, HI, USA: 47th Hawaii International Conference on System Sciences (HICSS). https://doi.org/10.1109/HICSS.2014.474

Jin, H., Yao, X., & Chen, Y. (2017). Correlation-aware QoS modeling and manufacturing cloud service composition. *Journal of Intelligent Manufacturing, 28*(8), 1947–1960. https://doi.org/10.1007/s10845-015-1080-2

Khanouche, M. E., Attal, F., Amirat, Y., Chibani, A., & Kerkar, M. (2019). Clustering-based and QoS-aware services composition algorithm for ambient intelligence. *Information Sciences, 482*, 419–439. https://doi.org/10.1016/j.ins.2019.01.015

Khurana, R., & Bawa, R. K. (2016). *QoS based cloud service selection paradigms* p. 174-179. Noida, India: 6th International Conference - Cloud System and Big Data Engineering (Confluence). https://doi.org/10.1109/CONFLUENCE.2016.7508109

Li, H., Chan, K. C., Liang, M., & Luo, X. (2016). Composition of resource-service chain for cloud manufacturing. *IEEE Transactions on Industrial Informatics, 12*(1), 211–219. https://doi.org/10.1109/TII.2015.2503126

Li, X., Ma, S., & Hu, J. (2017). Multi-search differential evolution algorithm. *Applied Intelligence, 47*(1), 231–256. https://doi.org/10.1007/s10489-016-0885-9

Liang, H., & Du, Y. (2017). Dynamic service selection with QoS constraints and inter-service correlations using cooperative coevolution. *Future Generation Computer Systems-the International Journal of eScience, 76*, 119–135. https://doi.org/10.1016/j.future.2017.05.019

Liang, Y., Xu, Q., & Jin, L. (2021). The effect of smart and connected products on consumer brand choice concentration. *Journal of Business Research, 135*, 163–172. https://doi.org/10.1016/j.jbusres.2021.06.039

de Melo, V. V., & Iacca, G. (2014). A modified covariance matrix adaptation evolution strategy with adaptive penalty function and restart for constrained optimization. *Expert Systems with Applications, 41*(16), 7077–7094. https://doi.org/10.1016/j.eswa.2014.06.032

Mo, Q., Wang, Y., Xiang, J., & Li, T. (2020). A correctness checking approach for collaborative business processes in the cloud. *Complexity, 2020*, 1–11. https://doi.org/10.1155/2020/2751084

Naseri, A., & Navimipour, N. J. (2019). A new agent-based method for QoS-aware cloud service composition using particle swarm optimization algorithm. *Journal of Ambient Intelligence and Humanized Computing, 10*(5), 1851–1864. https://doi.org/10.1007/s12652-018-0773-8

Nunez, M. A., Bai, X., & Du, L. (2021). Leveraging slack capacity in IaaS contract cloud services. *Production and Operations Management, 30*(4), 883–901. https://doi.org/10.1111/poms.13283

Opara, K. R., & Arabas, J. (2019). Differential evolution: A survey of theoretical analyses. *Swarm and Evolutionary Computation, 44*, 546–558. https://doi.org/10.1016/j.swevo.2018.06.010

Passacantando, M., Ardagna, D., & Savi, A. (2016). Service provisioning problem in cloud and multi-cloud systems. *INFORMS Journal on Computing, 28*(2), 265–277. https://doi.org/10.1287/ijoc.2015.0681

Patros, P., Spillner, J., Papadopoulos, A. V., Varghese, B., Rana, O., & Dustdar, S. (2021). Toward sustainable serverless computing. *IEEE Internet Computing, 25*(6), 42–50. https://doi.org/10.1109/mic.2021.3093105

Qi, J., Xu, B., Xue, Y., Wang, K., & Sun, Y. (2018). Knowledge based differential evolution for cloud computing service composition. *Journal of Ambient Intelligence and Humanized Computing, 9*(3), 565–574. https://doi.org/10.1007/s12652-016-0445-5

Rehman, Z.-U., Hussain, O. K., & Hussain, F. K. (2015). User-side cloud service management: State-of-the-art and future directions. *Journal of Network and Computer Applications, 55*, 108–122. https://doi.org/10.1016/j.jnca.2015.05.007

Scheepers, H., & Scheepers, R. (2008). A process-focused decision framework for analyzing the business value potential of it investments. *Information Systems Frontiers, 10*(3), 321–330. https://doi.org/10.1007/s10796-008-9076-5

Schulte, S., Janiesch, C., Venugopal, S., Weber, I., & Hoenisch, P. (2015). Elastic business process management: State of the art and open challenges for BPM in the cloud. *Future Generation Computer Systems-the International Journal of eScience, 46*, 36–50. https://doi.org/10.1016/j.future.2014.09.005

Tao, F., Hu, Y., Zhao, D., Zhou, Z., Zhang, H., & Lei, Z. (2009). Study on manufacturing grid resource service QoS modeling and evaluation. *The International Journal of Advanced Manufacturing Technology, 41*(9), 1034–1042. https://doi.org/10.1007/s00170-008-1534-1

Thakur, S., & Breslin, J. G. (2019). A robust reputation management mechanism in the federated cloud. *IEEE Transactions on Cloud Computing, 7*(3), 625–637. https://doi.org/10.1109/tcc.2017.2689020

Venkatesh, V., Morris, M. G., Davis, G. B., & Davis, F. D. (2003). User acceptance of information technology: Toward a unified view. *MIS Quarterly, 27*(3), 425–478. https://doi.org/10.5555/2017197.2017202

Waibel, P., Hochreiner, C., Schulte, S., Koschmider, A., & Mendling, J. (2021). Viepep-c: A container-based elastic process platform. *IEEE Transactions on Cloud Computing, 9*(4), 1657–1674. https://doi.org/10.1109/TCC.2019.2912613

Wu, Y., Jia, G., & Cheng, Y. (2020). Cloud manufacturing service composition and optimal selection with sustainability considerations: A multi-objective integer bi-level multi-follower programming approach. *International Journal of Production Research, 58*(19), 6024–6042. https://doi.org/10.1080/00207543.2019.1665203

Xu, J., Liang, C., Jain, H. K., & Gu, D. (2019). Openness and security in cloud computing service: Assessment methods and investment strategies analysis. *IEEE Access, 7*, 29038–29050. https://doi.org/10.1109/access.2019.2900889

Xue, X., Liu, Z.-Z., & Wang, S.-F. (2016). Manufacturing service composition for the mass customised production. *International Journal of Computer Integrated Manufacturing, 29*(2), 119–135. https://doi.org/10.1080/0951192x.2014.1002813

Yang, Y., Yang, B., Wang, S., Liu, F., Wang, Y., & Shu, X. (2019). A dynamic ant-colony genetic algorithm for cloud service composition optimization. *International Journal of Advanced Manufacturing Technology, 102*(1–4), 355–368. https://doi.org/10.1007/s00170-018-03215-7

Yoo, S.-K., & Kim, B.-Y. (2018). A decision-making model for adopting a cloud computing system. *Sustainability, 10*(8), 1–15. https://doi.org/10.3390/su10082952

Zhang, W., Guo, H., Zeng, Z., Qi, Y., & Wang, Y. (2018). Transportation cloud service composition based on fuzzy programming and genetic algorithm. *Transportation Research Record, 2672*(45), 64–75. https://doi.org/10.1177/0361198118796711

Zheng, Q., Gu, D., Liang, C., & Fang, Y. (2020). Impact of a firm's physical and knowledge capital intensities on its selection of a cloud computing deployment model. *Information & Management, 57*(7), 103,259. https://doi.org/10.1016/j.im.2019.103259

Zhou, J., & Yao, X. (2017). DE-caABC: differential evolution enhanced context-aware artificial bee colony algorithm for service composition and optimal selection in cloud manufacturing. *International Journal of Advanced Manufacturing Technology, 90*(1–4), 1085–1103. https://doi.org/10.1007/s00170-016-9455-x

Zhou, J., & Yao, X. (2017). A hybrid artificial bee colony algorithm for optimal selection of QoS-based cloud manufacturing service composition. *International Journal of Advanced Manufacturing Technology, 88*(9–12), 3371–3387. https://doi.org/10.1007/s00170-016-9034-1

Zhou, J., & Yao, X. (2017). Hybrid teaching-learning-based optimization of correlation-aware service composition in cloud manufacturing. *International Journal of Advanced Manufacturing Technology, 91*(9), 3515–3533. https://doi.org/10.1007/s00170-017-0008-8

Zhou, J., Yao, X., Lin, Y., Chan, F. T. S., & Li, Y. (2018). An adaptive multi-population differential artificial bee colony algorithm for many-objective service composition in cloud manufacturing. *Information Sciences, 456*, 50–82. https://doi.org/10.1016/j.ins.2018.05.009

Jian Xu is a senior lecturer and researcher of intelligent information systems, data science and heuristic optimization in School of Statistics and Applied Mathematics, Anhui University of Finance and Economics. He earned a Ph.D. degree in Information Management and Information Systems from the Hefei University of Technology, and has been working on cloud computing and data science. He served as reviewers for several international journals and conferences, and published papers in several IEEE conferences and journals.

Hemant K. Jain is W. Max Finely Chair and Professor of Data Analytics, in Rollins College of Business at The University of Tennessee Chattanooga. He is also Director of MS in Data Analytics program. He has published more than 100 articles in leading Information Systems and computer science journals such as ISR, MISQ, JMIS, IEEE Transactions on Software Engineering, IEEE Transactions on Systems Man and Cybernetics, Naval Research Quarterly, Decision Sciences, Decision Support Systems, Communications of ACM, Information & Management, and Information Systems Frontiers. He recently co-edited a special issue of ISR on Humans, Algorithms, and Augmented Intelligence: The Future of Work, Organizations, and Society. He received a best paper award from the European Journal of Information System. He also received IEEE Technical Committee on Service Computing's Outstanding Leadership Award. He served as Associate Editor-in-Chief of IEEE Transactions on Services Computing and as Associate Editor of JAIS & ISR. He received his Ph. D from Lehigh University, a M. Tech. from IIT Kharagpur, and B. E. University of Indore, India.

Dongxiao Gu is a professor of Management Information Systems at Hefei University of Technology. His research interests include Cloud Computing service, big data modeling based smart health, and business intelligence. His research has been published in Information & Management, Computers & Industrial Engineering, International Journal of Production Research, Information Processing & Management, Internet Research, IT & People, Applied Soft Computing, Expert Systems with Applications, Knowledge-Based Systems, etc.

Changyong Liang is a professor of Management Information Systems at Hefei University of Technology. His research interests cover information system management, cloud computing service and smart business. He is the author of over 120 publications in journals such as Information & Management, Computers & Industrial Engineering, Artificial Intelligence in Medicine, International Journal of Production Research, Applied Soft Computing, Expert Systems with Applications, Knowledge-Based Systems, International Journal of Intelligent Systems, etc.

Authors and Affiliations

Jian Xu[1,2] · Hemant K. Jain[3] · Dongxiao Gu[1,4] · Changyong Liang[1,5]

Jian Xu
jianx1982@163.com

Hemant K. Jain
hemant-jain@utc.edu

Changyong Liang
cyliang@163.com

1 School of Management, Hefei University of Technology, Hefei 230009, Anhui, China

2 School of Statistics and Applied Mathematics, Anhui University of Finance & Economics, Bengbu 233030, Anhui, China

3 Gary W. Rollins College of Business, The University of Tennessee at Chattanooga, Chattanooga, TN 37403, USA

4 Key Laboratory of Process Optimization and Intelligent Decision-Making, Ministry of Education, Hefei 230009, Anhui, China

5 Data Science and Smart Social Governance Lab, Hefei University of Technology, Hefei 230009, Anhui, China